AZURE AI / ML

PRACTITIONER HANDBOOK

Seventh Edition — The Definitive Azure AI Reference

For Python developers with no cloud experience → Azure AI/ML Practitioner

Subscription · Identity · Networking · Key Vault · OpenAI · AI Search · AI Foundry

Container Registry · Container Apps · App Service · Function App · API Management · Log Analytics

88 Visuals	9 Auth Types	5 AI Use Cases	Portal Walkthroughs	Complete Examples

Table of Contents

APPENDICES

PART 1: GETTING STARTED

Chapter 1: Azure Portal — Navigation & First Login

If you have never used Azure before, start here. This chapter covers how to navigate the Azure Portal, use Cloud Shell without installing anything locally, and understand which portal URL to use for which task.

1.1 The Azure Portal

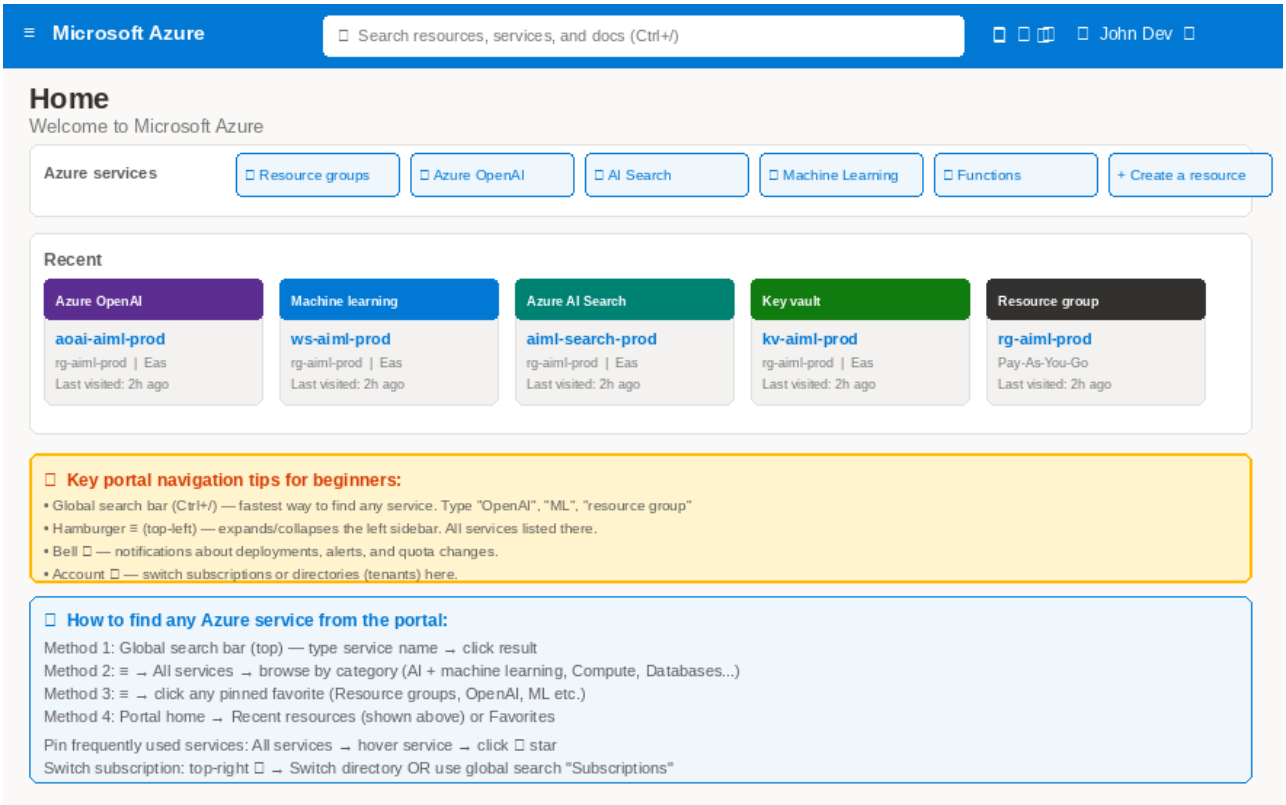


Fig 1.1 — Azure Portal home: search bar, recent resources, navigation and tips

The Azure Portal (portal.azure.com) is your browser-based control plane. Everything in Azure can be created, configured, and monitored here.

Element	Where to find it	What it does
Search bar (Ctrl+/)	Top centre of page	Fastest navigation — type any service name: "openai", "key vault", "resource group"
+ Create a resource	Home page or search results	Wizard to provision any new Azure service
≡ Hamburger menu	Top left corner	Expands left sidebar — All services organised by category
Bell 🔔 Notifications	Top right	Deployment complete, alerts firing, quota changes
Account 👤	Top right	Switch subscriptions, switch tenants (directories), sign out
Cloud Shell >_	Top right icon bar	Browser bash terminal — pre-authenticated, nothing to install
Favourites (sidebar)	Left navigation bar	Pin most-used services: star them in "All services"
Recent (home page)	Home page cards	Last resources visited — fastest way to return to work

1.2 Azure Portal vs AI Foundry — Which to Use When

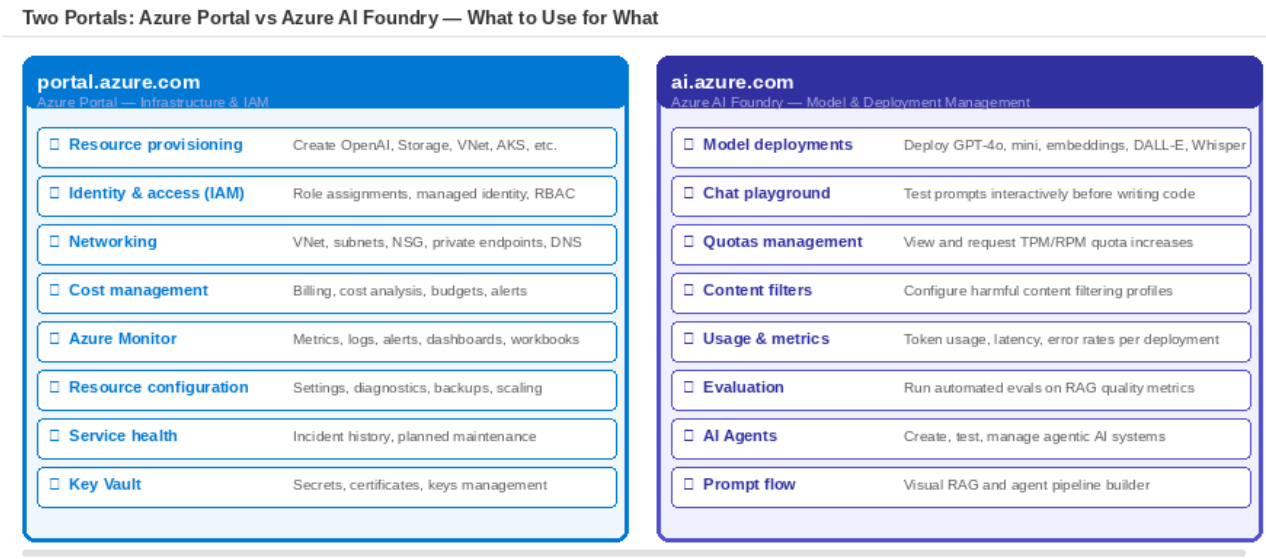


Fig 1.2 — Side-by-side comparison of Azure Portal vs Azure AI Foundry

If you want to…	Use this	URL
Create any resource (OpenAI, VNet, Storage, etc.)	Azure Portal	portal.azure.com → + Create a resource
Assign RBAC roles and manage identities	Azure Portal	Resource → Access control (IAM) → + Add role assignment
Configure networking, NSG, private endpoints	Azure Portal	Resource → Networking tab
View costs and set budget alerts	Azure Portal	Cost Management → Cost analysis
Deploy GPT-4o or text-embedding-3-large model	Azure AI Foundry	ai.azure.com → Deployments → + Deploy model
Test prompts before writing code	Azure AI Foundry	ai.azure.com → Chat playground
Request more TPM quota	Azure AI Foundry	ai.azure.com → Quotas → Request increase
Configure content safety filters	Azure AI Foundry	ai.azure.com → Content safety

1.3 Azure Cloud Shell

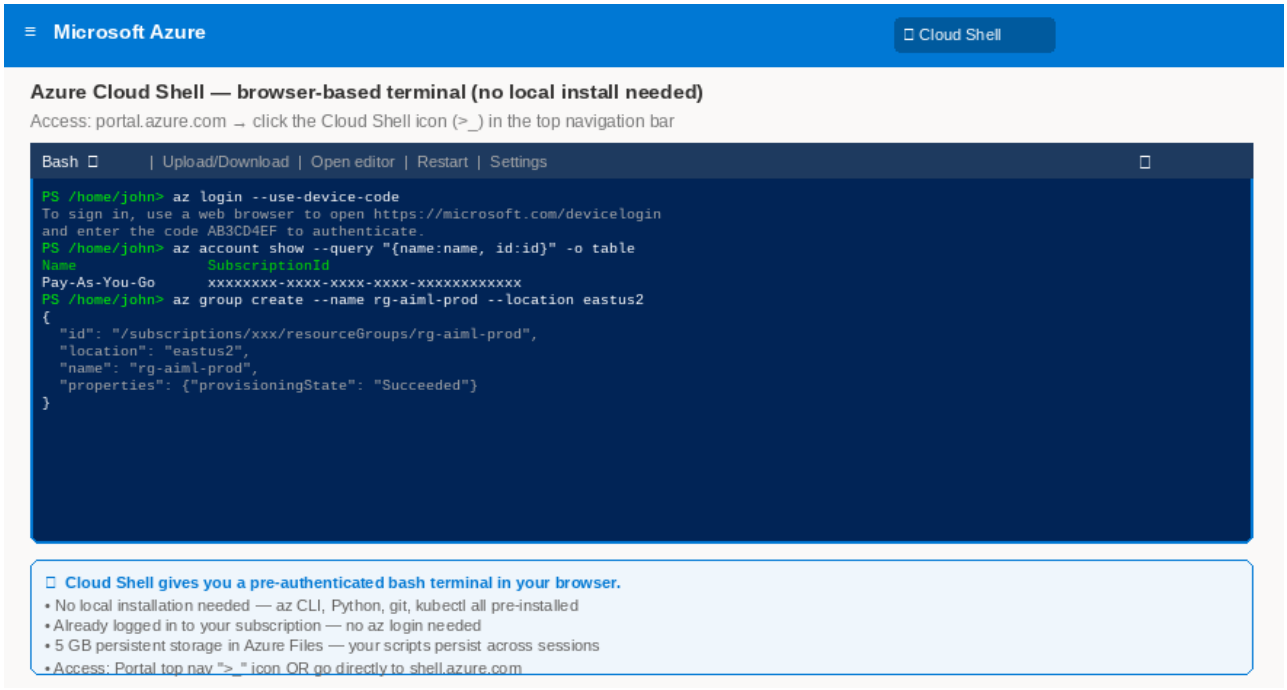


Fig 1.3 — Azure Cloud Shell: pre-authenticated browser bash terminal

Cloud Shell gives you a bash terminal in your browser. No local installation needed. It is already authenticated to your Azure subscription.

1. Open: portal.azure.com → click >_ icon in the top navigation bar → select Bash.
2. First time: Azure creates a storage account (5 GB) to persist your files between sessions.
3. Pre-installed: az CLI, Python 3, pip, git, kubectl, helm, terraform, jq, vim, curl.
4. Files: save scripts to ~/clouddrive — they persist across sessions.
5. Tip: go directly to shell.azure.com for faster access.

```
# Verify your setup — run these first in Cloud Shell
az account show --query "{name:name, sub:id, tenant:tenantId}" -o table

# Set defaults to save typing --resource-group and --location every time
az configure --defaults group=rg-aiml-prod location=eastus2

# Install Azure AI Python SDKs
pip install -q azure-identity openai azure-search-documents \
            azure-keyvault-secrets azure-storage-blob azure-ai-documentintelligence

# Test DefaultAzureCredential (works because Cloud Shell is pre-authenticated)
python3 -c "
from azure.identity import DefaultAzureCredential
tok = DefaultAzureCredential().get_token('https://management.azure.com/.default')
print('Auth OK — token length:', len(tok.token))
"
```

1.4 Day-1 Checklist

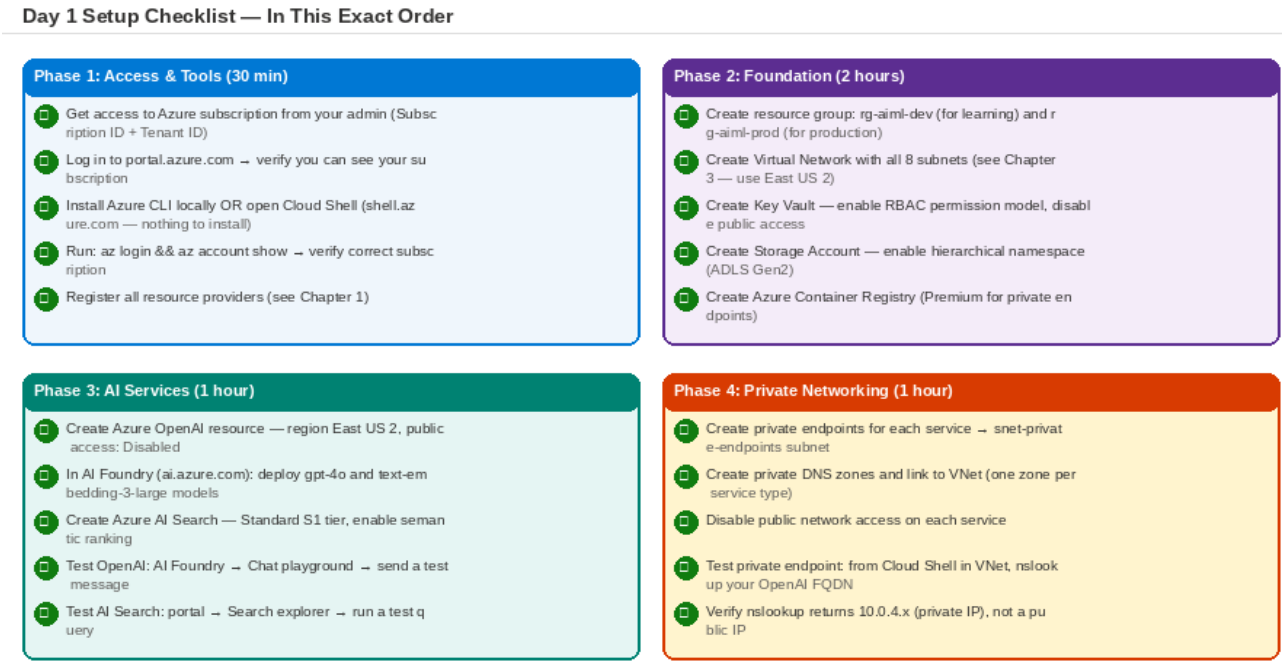


Fig 1.4 — Four-phase Day-1 setup checklist — follow in order

Chapter 2: Subscriptions, Tenants & Resource Groups

2.1 The Azure Hierarchy

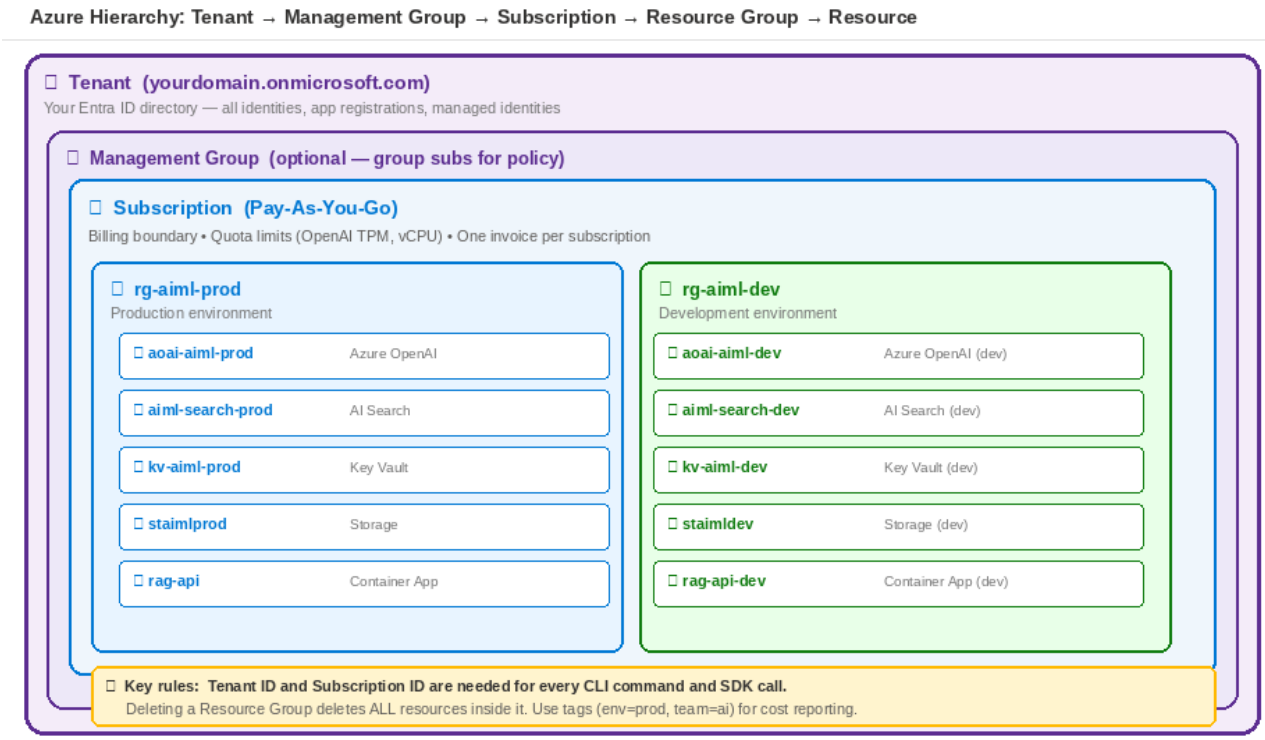


Fig 2.1 — Azure hierarchy: Tenant → Subscription → Resource Group → Resource

Key Terminology

Tenant (Entra ID)	Your company's Entra ID (formerly Azure AD) directory. All users, service principals, managed identities live here. One tenant per organisation. Get your Tenant ID: az account show --query tenantId	*Get it: az account show*
Subscription	A billing account that groups your resources. One invoice per subscription. Resource limits (OpenAI TPM quota, vCPU count) are scoped to subscription + region. Get your Subscription ID: az account show --query id	*Get it: az account show*
Resource Group (RG)	A logical container for resources that share the same lifecycle and environment. DELETING AN RG DELETES ALL RESOURCES INSIDE IT. Convention: rg-{project}-{env}	*Create: az group create*
Resource	A single Azure service instance — one OpenAI account, one Storage account, one Container App. Always lives inside an RG. Has an ARM Resource ID.	*List: az resource list*
ARM Resource ID	The unique path to any Azure resource: /subscriptions/{sub}/resourceGroups/{rg}/providers/{namespace}/{type}/{name}. Used in RBAC scope assignments and policy.	*Get: resource → JSON view*
Region	A geographic area (e.g. East US 2) hosting Azure data centres. Choose East US 2 for widest Azure OpenAI model availability.	*Recommended: eastus2*
Resource Provider	A namespace that enables a class of resources. Must be registered before you can create resources of that type. E.g. Microsoft.CognitiveServices for OpenAI.	*Register: az provider register*

2.2 Create Resource Group — Portal Walkthrough

1	**Navigate** Portal → type "Resource groups" in search bar → click Resource groups → + Create
2	**Fill Basics tab** Subscription: select yours | Name: rg-aiml-prod | Region: (US) East US 2
3	**Add tags** Tags tab: env = prod | team = ai-ml | cost-center = ai001 → Review + create → Create

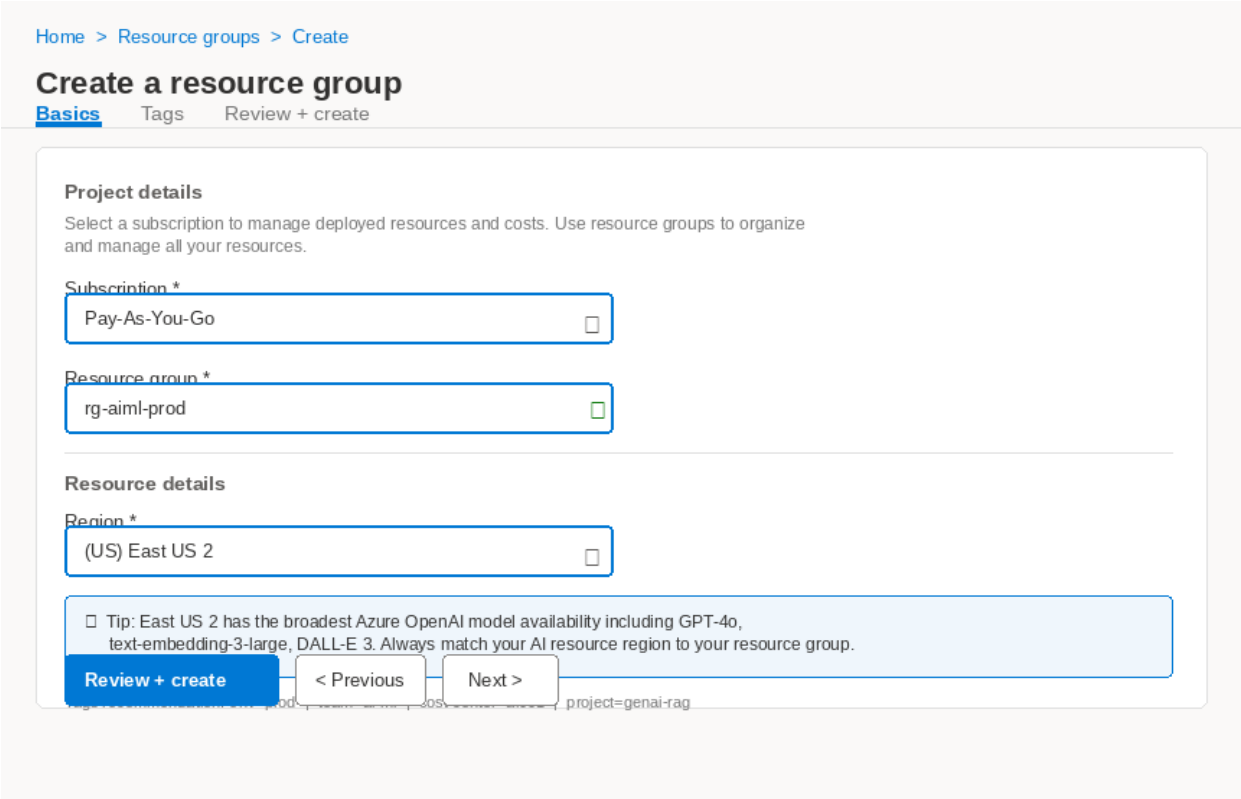


Fig 2.2 — Resource Group creation form with naming convention and tag guidance

2.3 Register Resource Providers

1	**Navigate** Portal → Subscriptions → click your subscription → Settings → Resource providers
2	**Register** Find each provider → click its name → click "Register" → wait ~1 minute → status shows "Registered"

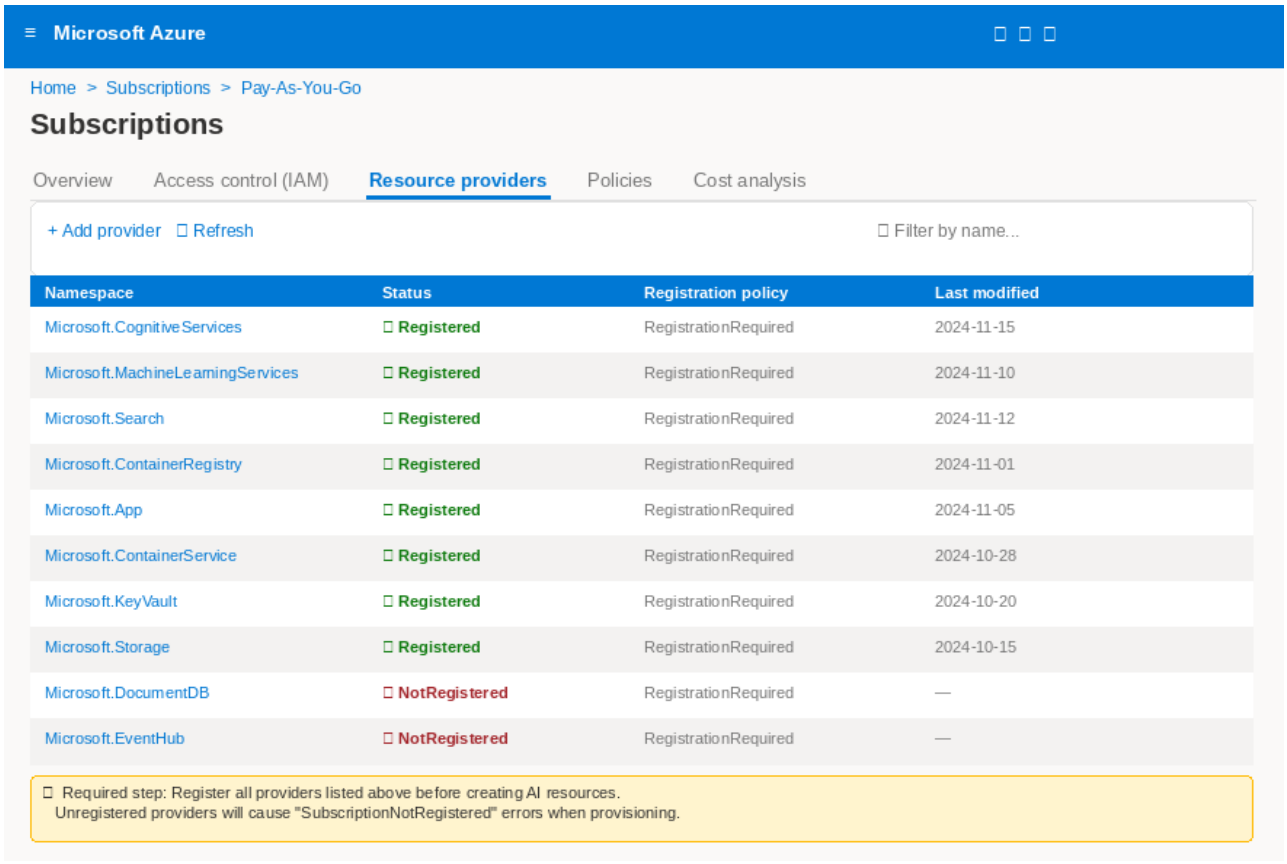


Fig 2.3 — Resource Providers page showing registered and unregistered providers

Provider	Register for	Status needed
Microsoft.CognitiveServices	Azure OpenAI, AI Vision, Speech, Document Intelligence	Registered
Microsoft.MachineLearningServices	Azure ML Workspace, compute clusters	Registered
Microsoft.Search	Azure AI Search	Registered
Microsoft.App	Container Apps and environments	Registered
Microsoft.ContainerRegistry	ACR — private image registry	Registered
Microsoft.KeyVault	Azure Key Vault	Registered
Microsoft.DocumentDB	Cosmos DB	Registered if using Cosmos
Microsoft.ServiceBus	Service Bus messaging	Registered if using SB

```
# Register all at once in Cloud Shell
for P in Microsoft.CognitiveServices Microsoft.MachineLearningServices \
         Microsoft.Search Microsoft.App Microsoft.ContainerRegistry \
         Microsoft.KeyVault Microsoft.DocumentDB Microsoft.ServiceBus; do
    az provider register --namespace $P && echo "Registered: $P"
done

# Verify — all should show "Registered"
az provider list --query "[?registrationState=='Registered'].namespace" -o tsv | sort | grep -E
"Cognitive|Machine|Search|App|Container|Vault|Document|Service"
```

PART 2: AUTHENTICATION — THE COMPLETE GUIDE

Chapter 3: Authentication — All 9 Types Explained

Authentication is the most critical concept to master in Azure. Getting it right means zero secrets in your code. Getting it wrong means security breaches or constant access failures. This chapter explains every authentication type in depth — what it is, how it works, and when to use it.

3.1 Key Terminology — OAuth 2.0, OIDC, JWT, Bearer Token

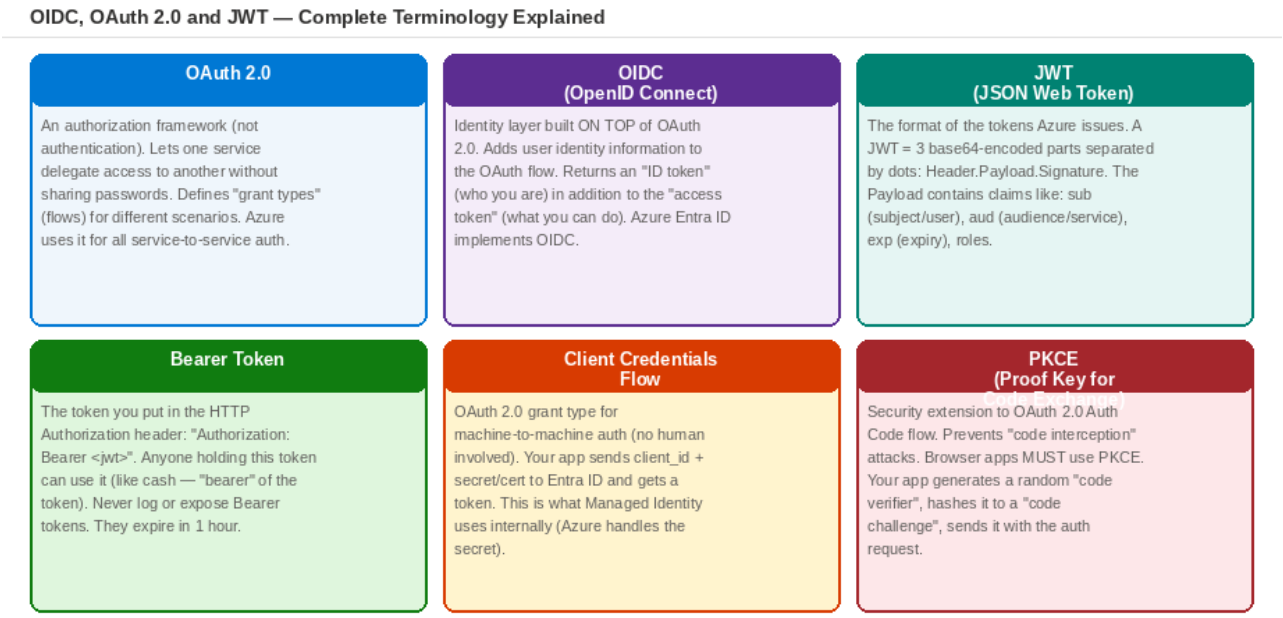


Fig 3.1 — OAuth 2.0, OIDC, JWT, Bearer Token, Client Credentials, PKCE — all explained

Understanding these terms helps you debug authentication failures and choose the right auth type:

OAuth 2.0	An authorization framework (not authentication). Defines how one service can delegate access to another without sharing passwords. Azure uses OAuth 2.0 for ALL service-to-service auth. Entra ID is the OAuth 2.0 authorization server.
OIDC (OpenID Connect)	Identity layer built ON TOP of OAuth 2.0. Adds user identity ("who are you") to OAuth. Returns an ID token alongside the access token. GitHub Actions uses OIDC to get tokens from Entra ID without needing a stored secret.
JWT (JSON Web Token)	The format of tokens issued by Entra ID. Three base64-encoded parts: Header (algorithm) + Payload (claims: who, what, when expires) + Signature (cryptographic proof). Your app never creates JWTs — Entra ID does.
Bearer Token	A JWT in the HTTP Authorization header: "Authorization: Bearer eyJ0eXAi...". Anyone holding it can use it. Tokens expire in 1 hour. The SDK auto-refreshes them. NEVER log or expose Bearer tokens.
Client Credentials Flow	OAuth 2.0 grant type for machine-to-machine auth with no human user involved. App sends client_id + secret/cert to Entra ID → gets access token. Managed Identity uses this flow internally — Azure manages the credential for you.
PKCE (Proof Key for Code	Security extension for OAuth 2.0 Auth Code flow. Prevents "code interception" attacks. Required for all browser/mobile apps. App generates a random code_verifier, hashes it as code_challenge,

Exchange)	sends both as proof of identity.
Scope	What the token grants access to. Format: "https://cognitiveservices.azure.com/.default" means full access to Azure Cognitive Services. Scopes are listed in the app registration.
Federated Credential (OIDC federation)	A trust relationship between a platform (GitHub, AKS) and an App Registration. GitHub Actions can get an Entra ID token by presenting its own OIDC token — no client secret needed at all.

3.2 Authentication Types — Overview & Decision Guide

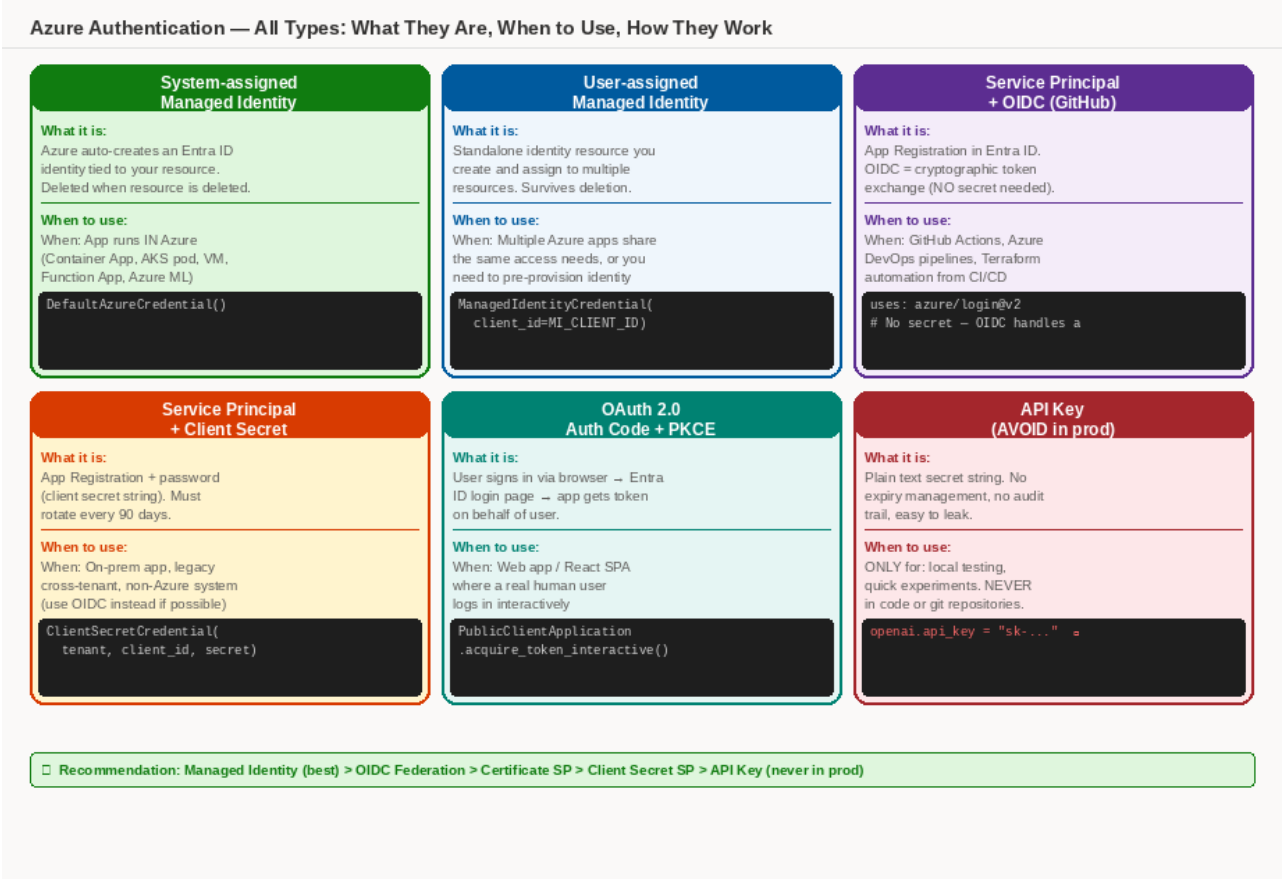


Fig 3.2 — All 6 authentication types: what they are, when to use, code example

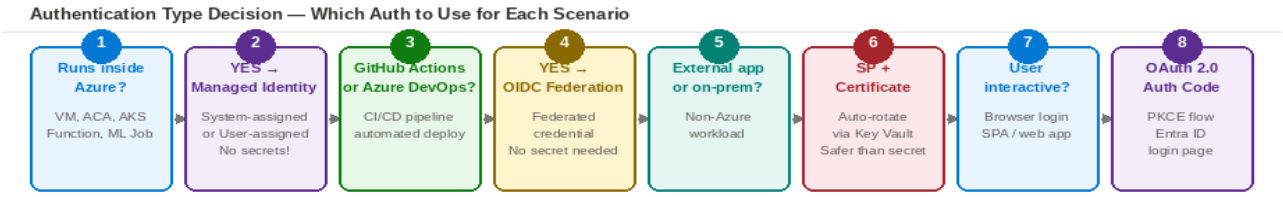


Fig 3.3 — Decision flowchart: which auth type to choose for each scenario

Auth type	When to use	Secret management	Security rating
System-assigned MI	App runs IN Azure (ACA, AKS pod, VM, Functions)	None — Azure handles everything	★★★★★ Best
User-assigned MI	Multiple Azure apps share access, pre-provision identity	None — Azure handles everything	★★★★★ Best
OIDC Federation	GitHub Actions, Azure DevOps, AKS workload identity	None — cryptographic exchange	★★★★★ Best

Auth type	When to use	Secret management	Security rating
SP + Certificate	On-prem app calling Azure, cross-tenant access	Certificate (auto-rotate via KV)	★★★★ Good
SP + Client Secret	Legacy CI/CD, non-Azure (use OIDC instead)	String password (rotate every 90d)	★★★ OK
OAuth 2.0 + PKCE	User signs in via browser (SPA, web app)	User credential in Entra ID	★★★★ Good
Client Credentials	Background service calls another service (no user)	Client secret or certificate	★★★ OK
API Key	NEVER in production — local experiments only	You manage manually	★ Avoid

3.3 Type 1: System-assigned Managed Identity

The most common auth type for Azure workloads. Azure creates a service principal in Entra ID for your resource and rotates its credentials automatically. Your code uses DefaultAzureCredential() — no config needed.

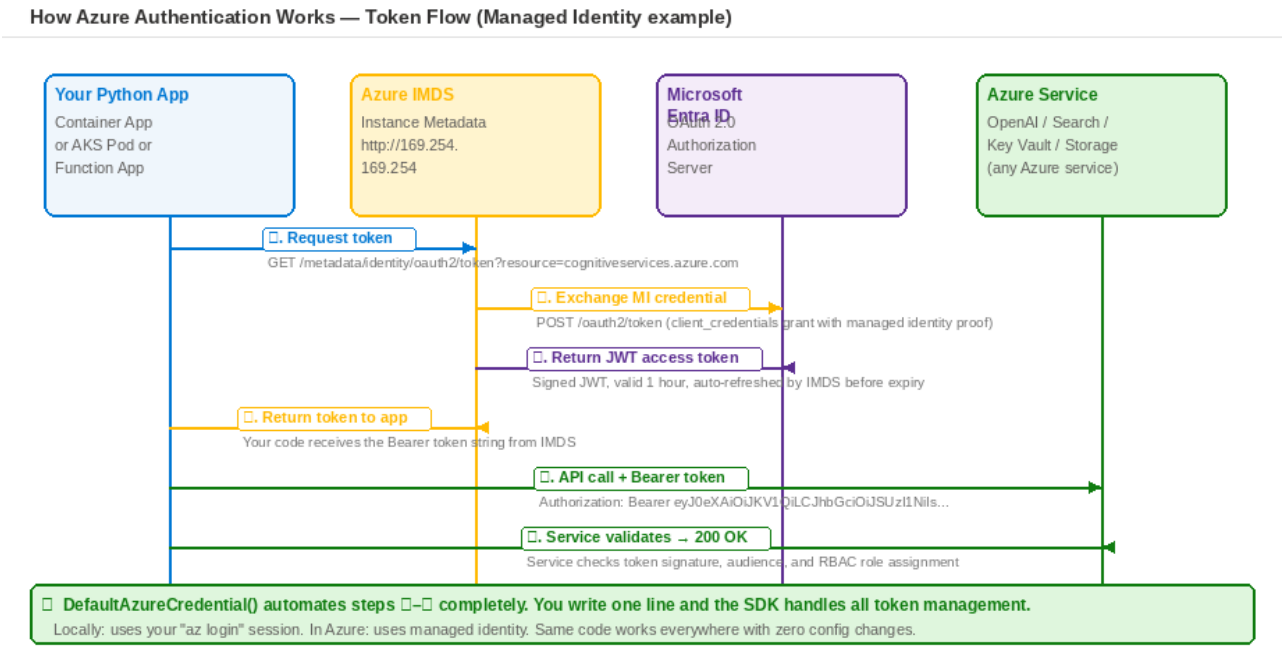


Fig 3.4 — Token flow: how DefaultAzureCredential() gets a token for your app automatically

How it works — step by step

6. You enable system-assigned MI on your Container App / AKS pod / VM / Function.
7. Azure creates a service principal in Entra ID and stores its credential in IMDS (Instance Metadata Service).
8. When DefaultAzureCredential() is called, it contacts IMDS at http://169.254.169.254 (internal Azure IP).
9. IMDS exchanges the managed identity credential with Entra ID and returns a JWT access token.
10. The SDK attaches the token to your API call: Authorization: Bearer <jwt>.
11. The target service (OpenAI, Search, etc.) validates the token and checks your RBAC role assignment.
12. Token is cached and auto-refreshed before expiry — you never manage tokens yourself.

1	**Enable MI on your resource** Portal → Container Apps → your app → Identity (left sidebar) → System assigned tab → toggle Status to On → Save

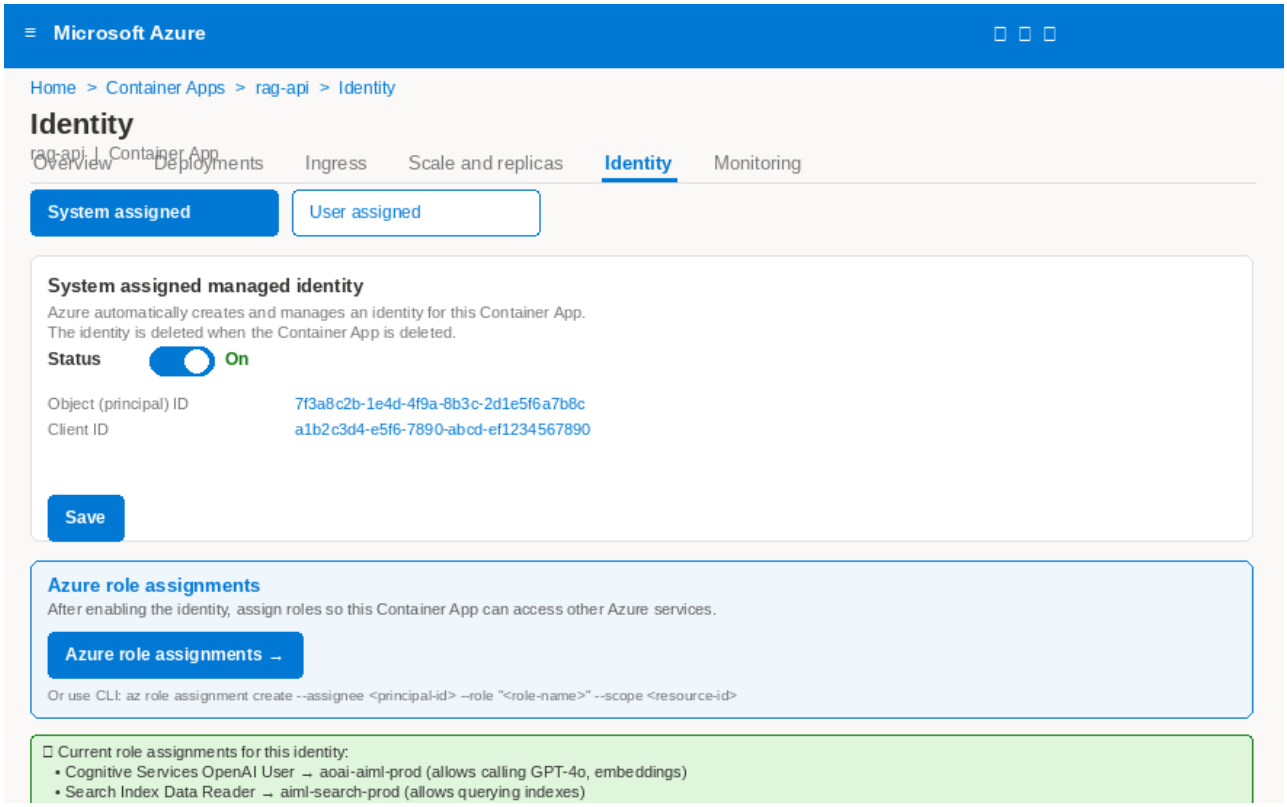


Fig 3.5 — Container App Identity tab: enable system-assigned MI, view Principal ID, click Azure role assignments

2	**Copy the Principal ID** After saving: copy the Object (principal) ID shown — you will use this for role assignments
3	**Assign RBAC role** Click "Azure role assignments" button → Add role assignment → fill in Role, Members, Scope

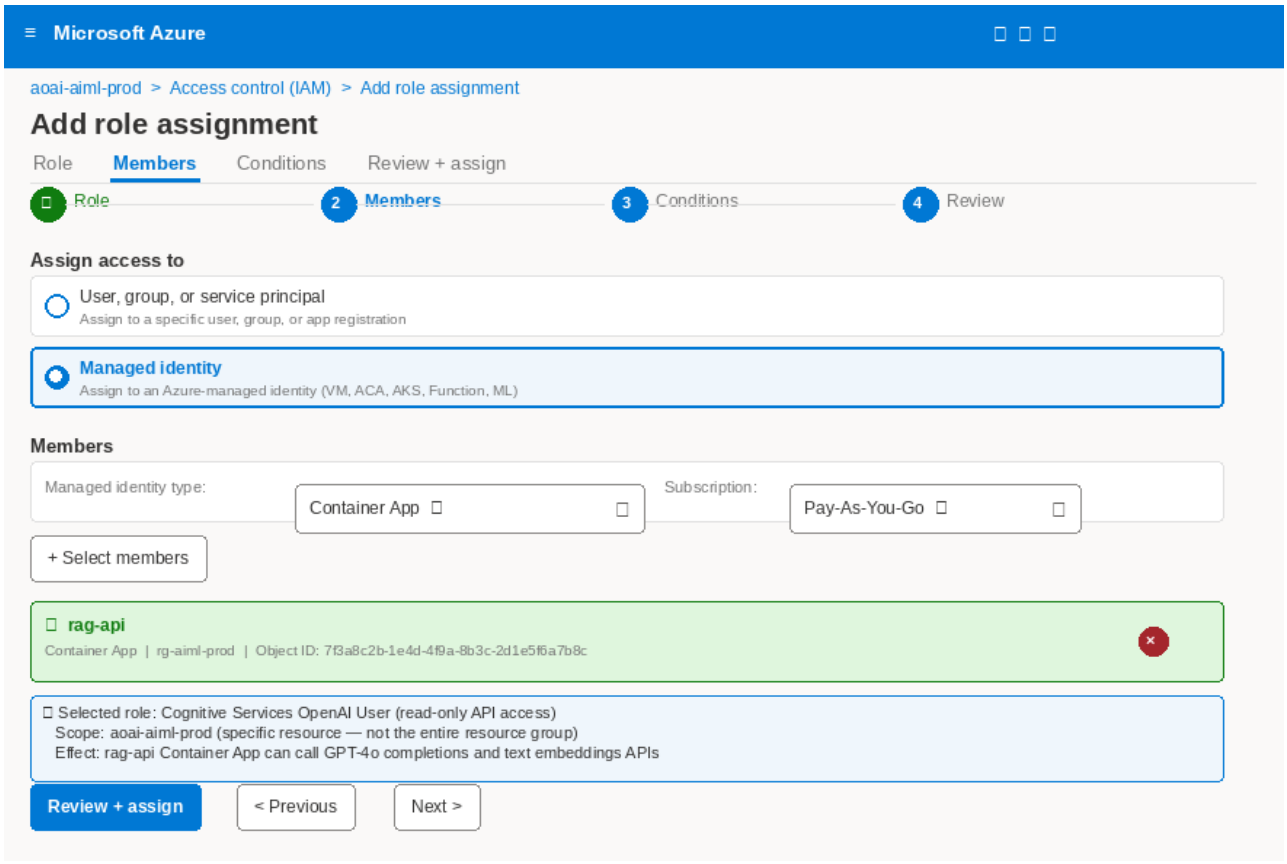


Fig 3.6 — Add role assignment: select Managed identity, choose your app, verify scope is specific resource

Role assignment — click-by-click

13. Role tab: search for "Cognitive Services OpenAI User" → select → Next.
14. Members tab: "Assign access to" → choose "Managed identity" (never User or SP for Azure apps).
15. Members tab: Managed identity type → "Container Apps" → click "+ Select members" → find your app.
16. Review + assign tab: verify Scope shows the SPECIFIC RESOURCE (not subscription or RG) → click Review + assign.
17. Wait 2-5 minutes for RBAC to propagate before testing.

```
# Enable MI and assign all roles for a RAG API — run in Cloud Shell
RG="rg-aiml-prod"; APP="rag-api"

# Enable system-assigned MI on Container App
az containerapp identity assign -n $APP -g $RG --system-assigned

# Get Principal ID
MI=$(az containerapp show -n $APP -g $RG --query identity.principalId -o tsv)
echo "Principal ID: $MI"

# Get resource IDs for each service
AOAI=$(az cognitiveservices account show -n aoai-aiml-prod -g $RG --query id -o tsv)
SRCH=$(az search service show -n aiml-search-prod -g $RG --query id -o tsv)
KV=$(az keyvault show -n kv-aiml-prod -g $RG --query id -o tsv)
SA=$(az storage account show -n staimlprod -g $RG --query id -o tsv)

# Assign minimum required roles — least privilege principle
```

```
az role assignment create --role "Cognitive Services OpenAI User" --assignee $MI --scope $AOAI
az role assignment create --role "Search Index Data Reader"       --assignee $MI --scope $SRCH
az role assignment create --role "Key Vault Secrets User"         --assignee $MI --scope $KV
az role assignment create --role "Storage Blob Data Reader"       --assignee $MI --scope $SA

# Verify assignments
az role assignment list --assignee $MI --all -o table

# Python code — DefaultAzureCredential detects MI automatically when running in Azure
from azure.identity import DefaultAzureCredential
from azure.identity import get_bearer_token_provider
from openai import AzureOpenAI
import os

cred = DefaultAzureCredential()   # no config needed — auto-detects MI

client = AzureOpenAI(
    azure_endpoint=os.environ["OPENAI_ENDPOINT"],
    azure_ad_token_provider=get_bearer_token_provider(
        cred, "https://cognitiveservices.azure.com/.default"),
    api_version="2024-08-01-preview"
)
```

3.4 Type 2: User-assigned Managed Identity

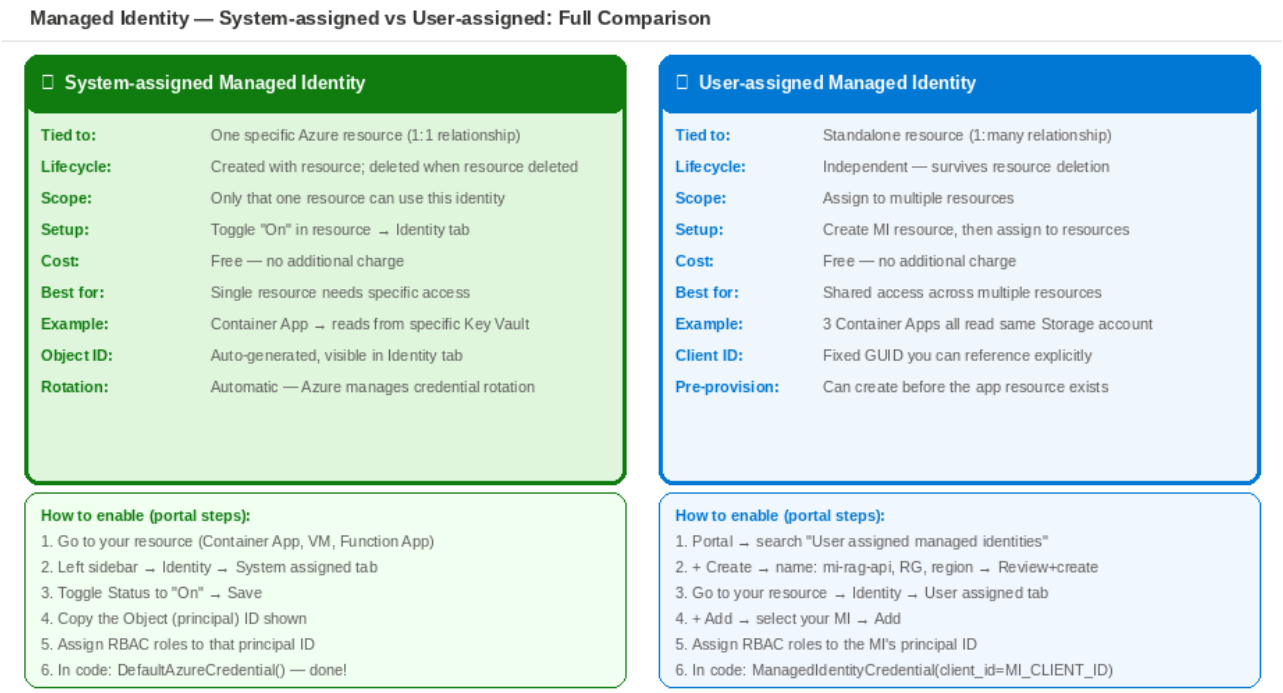


Fig 3.7 — System-assigned vs User-assigned MI: properties, lifecycle, and portal setup steps

```
# Create user-assigned MI
az identity create --name mi-rag-shared -g rg-aiml-prod --location eastus2

# Get BOTH IDs — you need different ones for different purposes
```

```
MI_CLIENT=$(az identity show -n mi-rag-shared -g rg-aiml-prod --query clientId -o tsv)
MI_OBJECT=$(az identity show -n mi-rag-shared -g rg-aiml-prod --query principalId -o tsv)
# principalId = for RBAC role assignments
# clientId   = for SDK calls when you want to specify which MI

# Assign roles (use principalId/objectId for RBAC, not clientId)
az role assignment create --role "Cognitive Services OpenAI User" \
    --assignee $MI_OBJECT --scope $AOAI_ID

# Assign MI to your Container App
az containerapp identity assign -n rag-api -g rg-aiml-prod \
    --user-assigned $(az identity show -n mi-rag-shared -g rg-aiml-prod --query id -o tsv)

# Python: explicitly specify which MI when you have multiple
from azure.identity import ManagedIdentityCredential
cred = ManagedIdentityCredential(client_id=MI_CLIENT_ID)
```

3.5 Type 3: App Registration & Service Principal

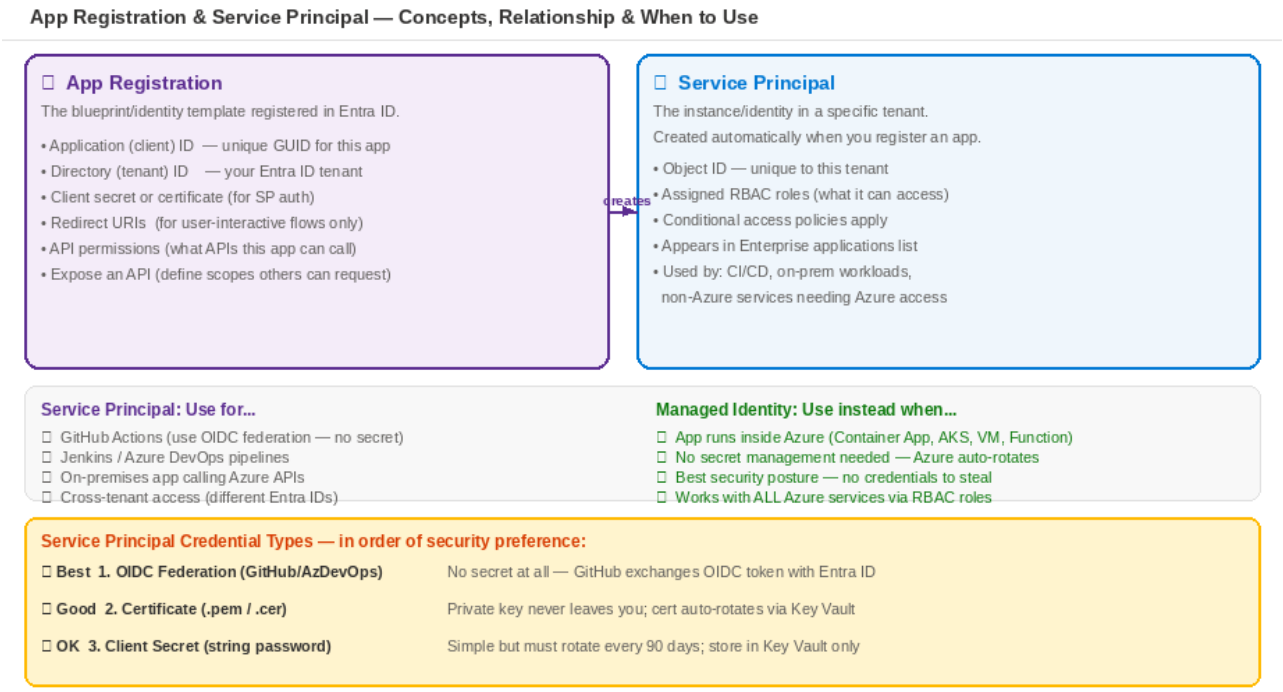


Fig 3.8 — App Registration vs Service Principal: what each is, credential types by security

1	**Create App Registration** Entra ID → App registrations → + New registration → name, single tenant → Register

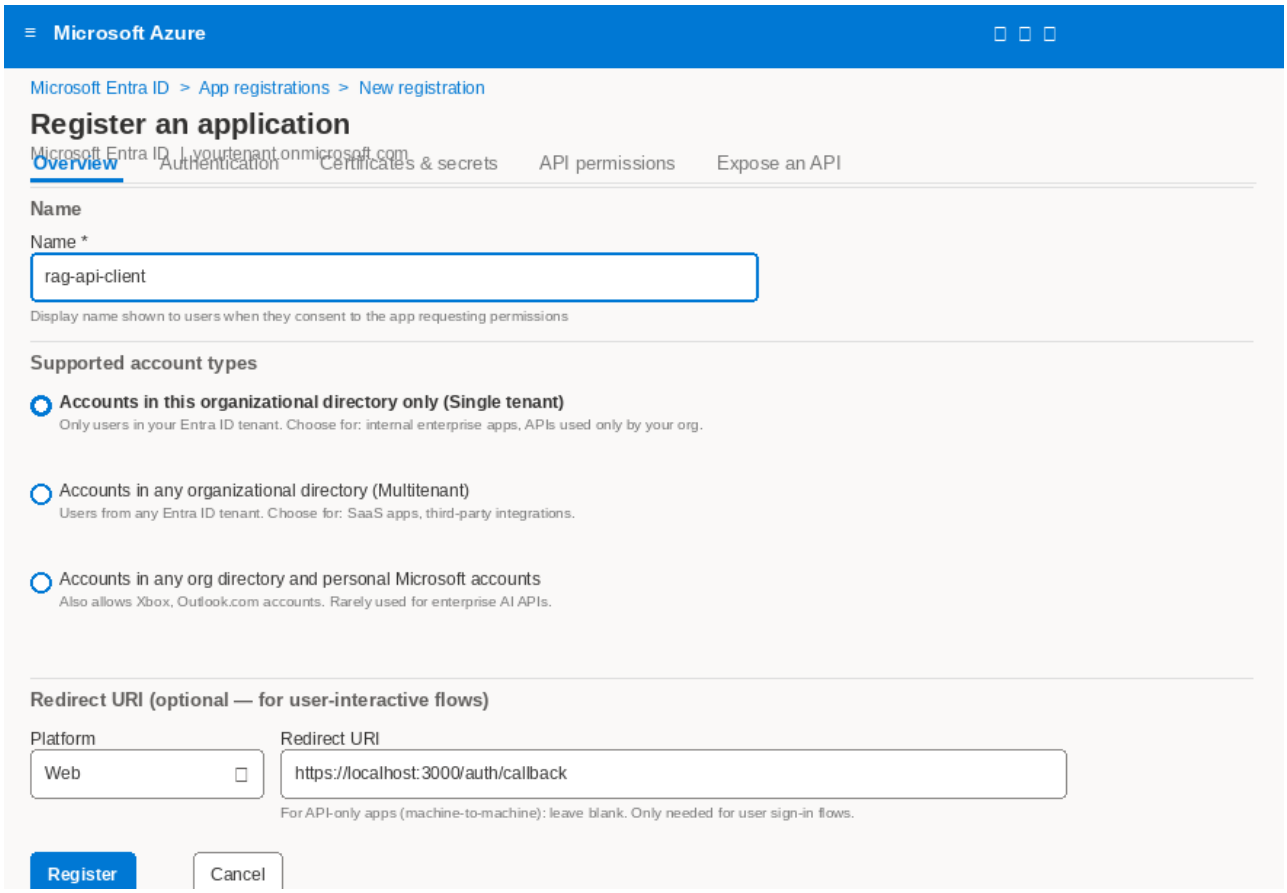


Fig 3.9 — App Registration form: name, account types, redirect URI (blank for CI/CD)

2	**Copy IDs** Overview page: copy Application (client) ID and Directory (tenant) ID — save both
3	**Create credential** Certificates & secrets → + New client secret → description, 12 months → Add → COPY VALUE NOW

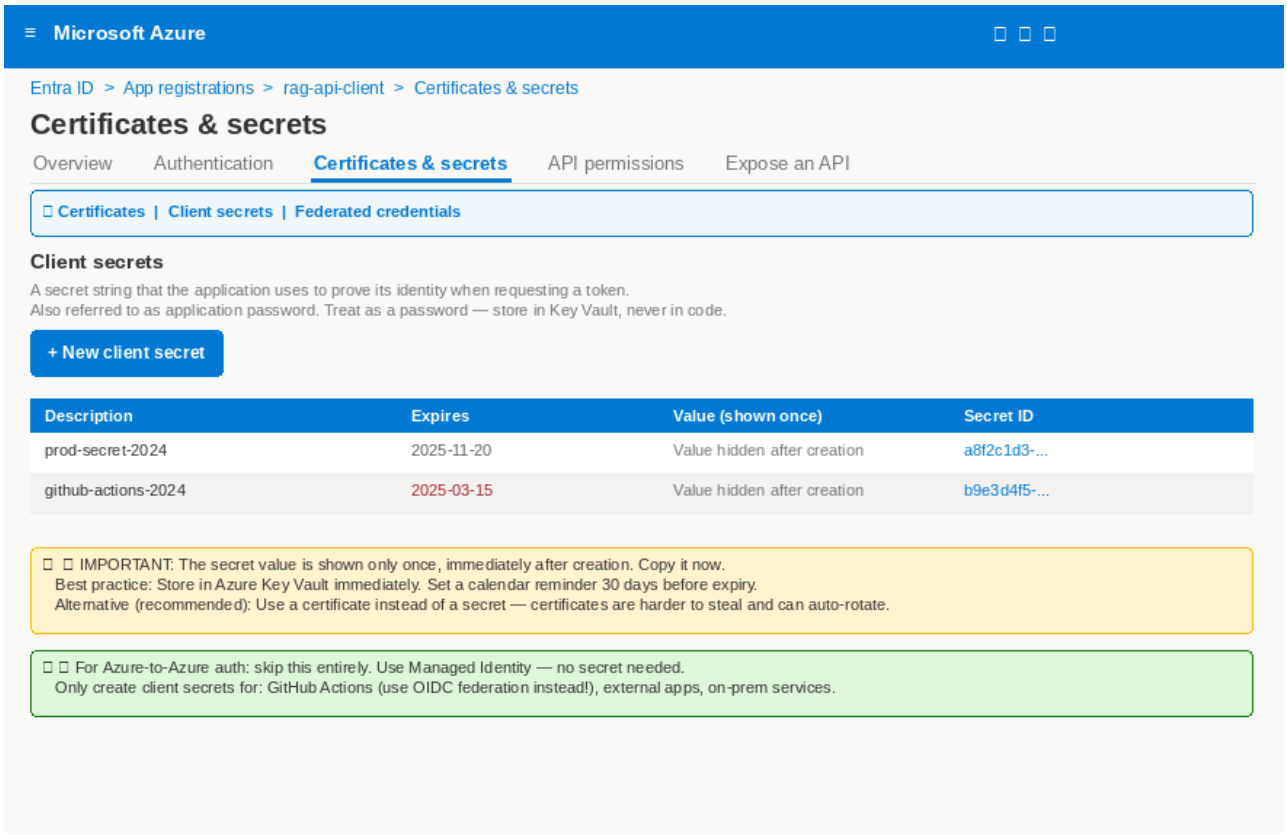


Fig 3.10 — Certificates & secrets page: one-time copy warning, Key Vault best practice

✓ For GitHub Actions: use Workload Identity Federation — eliminates client secrets entirely. App Registration → Certificates & secrets → Federated credentials → Add → GitHub Actions → enter your org/repo/branch. GitHub gets an Entra ID token by presenting its own OIDC token. No client secret ever created.

```
# Configure GitHub Actions OIDC federation (no client secret needed)

# 1. Create federated credential on App Registration
az ad app federated-credential create \
    --id <app-registration-object-id> \
    --parameters '{
        "name": "github-main-branch",
        "issuer": "https://token.actions.githubusercontent.com",
        "subject": "repo:your-org/your-repo:ref:refs/heads/main",
        "audiences": ["api://AzureADTokenExchange"]
    }'

# 2. Store IDs as GitHub secrets (no client secret!)
gh secret set AZURE_CLIENT_ID       --body "<Application (client) ID>"
gh secret set AZURE_TENANT_ID       --body "<Directory (tenant) ID>"
gh secret set AZURE_SUBSCRIPTION_ID --body "<Subscription ID>"

# 3. GitHub Actions workflow
permissions:
  id-token: write   # REQUIRED for OIDC
  contents: read

steps:
```

```
  - uses: azure/login@v2
    with:
      client-id: ${{ secrets.AZURE_CLIENT_ID }}
      tenant-id: ${{ secrets.AZURE_TENANT_ID }}
      subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
      # No client-secret field — OIDC handles it cryptographically
```

3.6 Type 4: OAuth 2.0 Auth Code + PKCE (User Login)

For web apps and SPAs where real users sign in with their Entra ID (work) account. The user authenticates via Entra ID's login page. Your app receives a token on behalf of that user.

```
# Backend FastAPI: validate the incoming JWT from a logged-in user
import httpx, os
from jose import jwt, JWTError
from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer

TENANT_ID = os.environ["AZURE_TENANT_ID"]
CLIENT_ID = os.environ["AZURE_CLIENT_ID"]  # Your API's App Registration

async def get_jwks():
    """Fetch Entra ID public keys for JWT signature verification"""
    async with httpx.AsyncClient() as h:
        oidc = await h.get(f"https://login.microsoftonline.com/{TENANT_ID}/v2.0/.well-known/
openid-configuration")
        jwks = await h.get(oidc.json()["jwks_uri"])
        return jwks.json()

async def validate_token(token: str = Depends(HTTPBearer())) -> dict:
    """Validate JWT and extract claims (user info)"""
    try:
        payload = jwt.decode(
            token.credentials,
            key=await get_jwks(),
            algorithms=["RS256"],
            audience=CLIENT_ID  # MUST match the "aud" claim in the token
        )
        return {
            "user_id": payload["oid"],      # Entra ID object ID of the user
            "email":   payload.get("preferred_username"),
            "name":    payload.get("name"),
            "roles":   payload.get("roles", []),
        }
    except JWTError as e:
        raise HTTPException(status_code=401, detail=f"Invalid token: {e}")

@app.post("/api/rag/query")
async def rag_query(body: QueryRequest, user=Depends(validate_token)):
    # user["user_id"] is the Entra ID object ID of the logged-in person
    return await do_rag(body.question, tenant_id=user["user_id"])
```

3.7 RBAC Roles Reference

RBAC Role Assignment Reference — Minimum Required Roles for Each Service

Rule: Always assign the MINIMUM role needed. Never use Owner or Contributor for application identities.

Service	Role name	What it allows	Assign to
Azure OpenAI	Cognitive Services OpenAI User	Call completions, embeddings, assistants APIs	App identity (read-only usage)
Azure OpenAI	Cognitive Services OpenAI Contributor	Also deploy/delete model versions	CI/CD pipelines
AI Search	Search Index Data Reader	Query indexes (search calls)	App that does search
AI Search	Search Index Data Contributor	Upsert/delete documents in index	Ingestion pipeline identity
AI Search	Search Service Contributor	Create/delete indexes, skillsets	IaC pipeline (one-time setup)
Storage	Storage Blob Data Reader	Read blobs (download documents)	App that reads documents
Storage	Storage Blob Data Contributor	Read + write blobs	Ingestion pipeline, ML training
Key Vault	Key Vault Secrets User	Read secret values	All application identities
Key Vault	Key Vault Secrets Officer	Create/update/delete secrets	CI/CD pipelines, admins
Container Registry	AcrPull	Pull container images	AKS, Container Apps, Functions
Container Registry	AcrPush	Push/build images to registry	CI/CD build pipelines
Document Intelligence	Cognitive Services User	Analyse documents	Ingestion function/app
Service Bus	Azure Service Bus Data Sender	Send messages to queues/topics	Producer apps
Service Bus	Azure Service Bus Data Receiver	Receive/peek messages	Consumer apps/functions
Cosmos DB	Cosmos DB Built-in Data Reader	Read documents	Read-only app queries
Cosmos DB	Cosmos DB Built-in Data Contributor	Read + write documents	App full access

□ Scope levels (from broadest to narrowest): Management Group > Subscription > Resource Group > Specific Resource

Fig 3.11 — Complete RBAC role reference for all Azure AI services

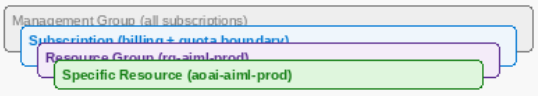
RBAC (Role-Based Access Control) — Concepts, Scope Levels & Common Roles

What is RBAC?

Role-Based Access Control is Azure's authorization system.
A role assignment has exactly 3 parts:

WHO (principal): a user, group, service principal, or managed identity
WHAT (role): a named set of permissions, e.g. "Storage Blob Data Reader"
WHERE (scope): Management Group > Subscription > Resource Group > specific Resource

Scope Levels (broad → narrow)



□ Assign to "Specific Resource" for all app identities

Most Important RBAC Roles for AI Solutions

Service	Role Name	What it allows	Assign to
Azure OpenAI	Cognitive Services Ope	Call completions + embeddings (read-on	Your RAG API identit
Azure AI Search	Search Index Data Read	Query / search index	App that does search
Azure AI Search	Search Index Data Cont	Upsert + delete index documents	Ingestion pipeline i
Key Vault	Key Vault Secrets User	Read secret values	All application iden
Storage	Storage Blob Data Read	Download / read blobs	App that reads docum
Storage	Storage Blob Data Cont	Read + write blobs	Ingestion pipeline i
Container Regist	AcrPull	Pull container images from registry	AKS kubelet, Contain
Document Intelli	Cognitive Services Use	Analyse documents (extract text)	Ingestion function /
Service Bus	Azure Service Bus Data	Send messages to queues/topics	Producer application

Fig 3.12 — RBAC concepts: what it is, scope levels, most important roles

PART 3: NETWORKING

Chapter 4: VNet, Subnets, NSG & Private Endpoints

4.1 VNet Architecture

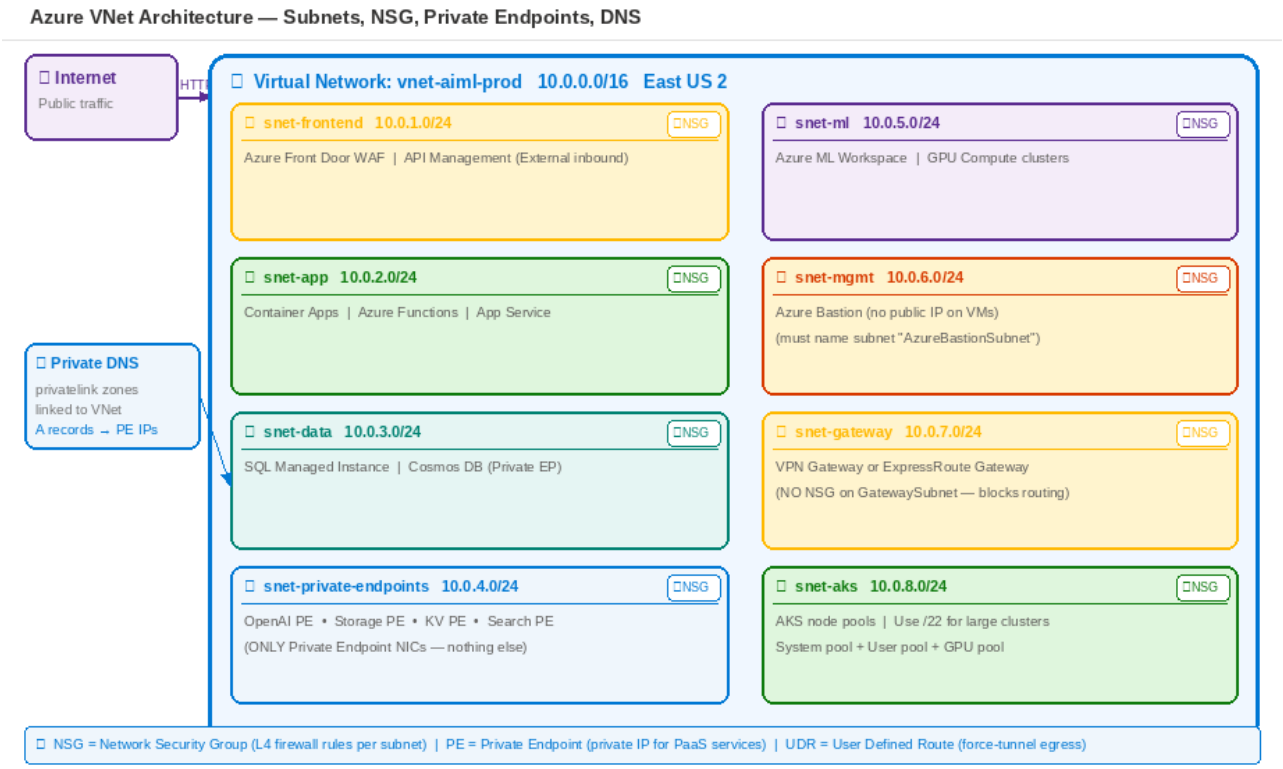


Fig 4.1 — VNet architecture: 8 subnets for the AI stack, NSG on each, Private DNS

Key Networking Terminology

VNet (Virtual Network)	Isolated software-defined network in Azure. You define the IP address space (e.g. 10.0.0.0/16 = 65,534 IPs). All resources inside share a private namespace and can communicate without internet.
Subnet	A subdivision of the VNet CIDR (e.g. 10.0.2.0/24 = 251 usable IPs). Resources are placed in subnets. NSGs are applied per subnet. Minimum /29 — Azure reserves 5 IPs per subnet.
NSG (Network Security Group)	Stateful Layer 4 firewall attached to a subnet or NIC. Rules specify source, destination, port, protocol, and allow/deny. Lower priority number = evaluated first. Default deny-all at 65500.
Service Tag	Named group of Azure service IP ranges maintained by Microsoft (e.g. "ApiManagement", "AzureMonitor", "Storage"). Use in NSG rules instead of hardcoding IP ranges that change.
Private Endpoint (PE)	A private IP address in YOUR VNet that maps to a specific PaaS service. Traffic to that IP stays entirely on the Azure backbone — never leaves your network. Requires a Private DNS Zone.
Private DNS Zone	An Azure DNS zone (e.g. privatelink.openai.azure.com) linked to your VNet. Resolves private endpoint FQDNs to their private IPs instead of public IPs. One zone per service type.
UDR (User Defined Route)	Custom route table entry that overrides Azure default routing. Common use: route 0.0.0.0/0 through Azure Firewall for egress inspection.

4.2 Create VNet — Portal Walkthrough

1	**Navigate** Portal → search "Virtual networks" → + Create → Basics: name=vnet-aiml-prod, region=East US 2
2	**IP Addresses tab** Address space: 10.0.0.0/16 → click "+ Add subnet" for each subnet below

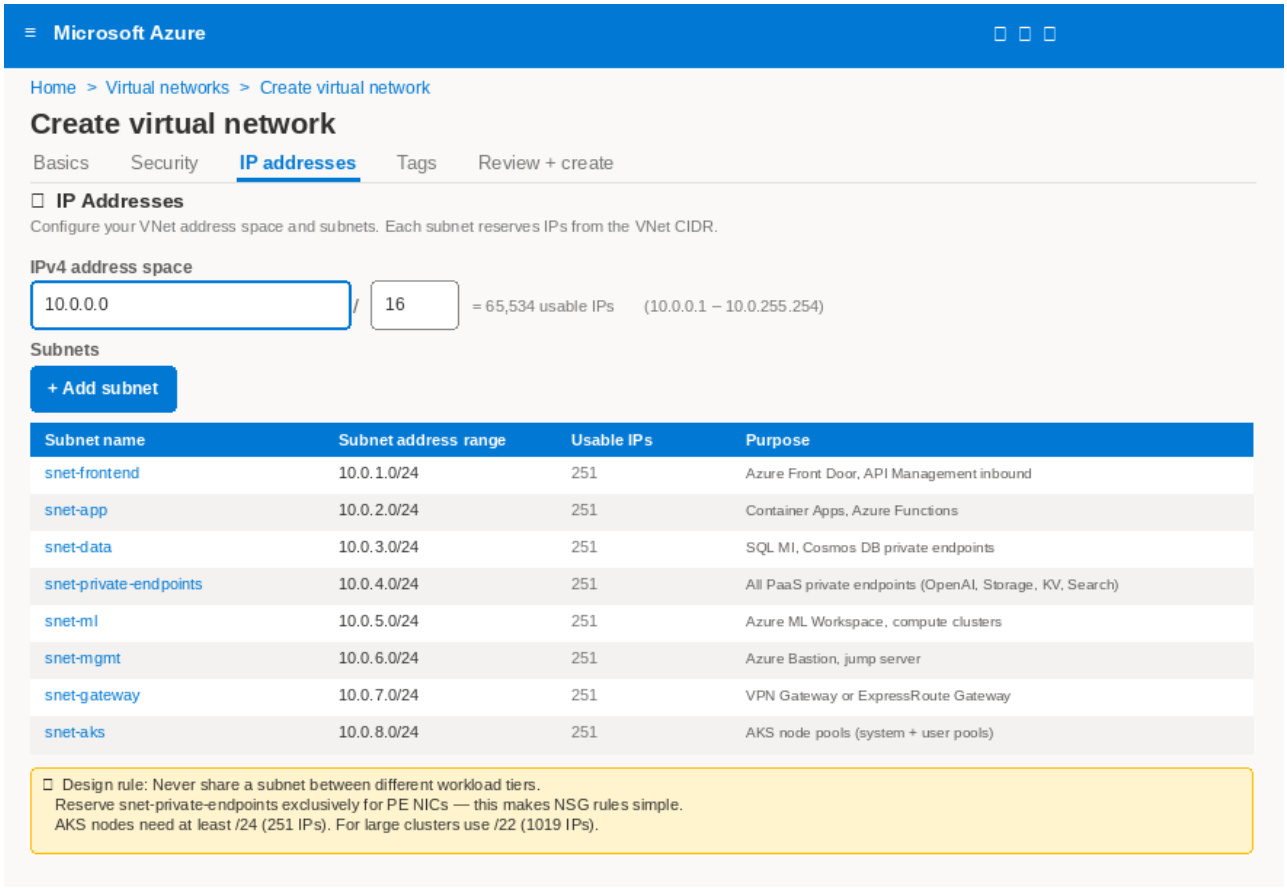


Fig 4.2 — VNet IP Addresses tab: 8 subnets with CIDR ranges and purposes

Subnet	CIDR	Hosts here	NSG rule summary
snet-frontend	10.0.1.0/24	Front Door origin, API Management	Allow 443 inbound from Internet tag
snet-app	10.0.2.0/24	Container Apps, Functions, App Service	Allow 443 from ApiManagement only
snet-data	10.0.3.0/24	SQL MI, Cosmos DB private endpoints	Allow 1433/5432 from snet-app only
snet-private-endpoints	10.0.4.0/24	ONLY PE NICs (OpenAI, KV, Search, Storage)	No inbound needed — PE ignores NSG
snet-ml	10.0.5.0/24	Azure ML workspace, GPU compute	Allow 443 outbound to PE subnet
snet-mgmt	10.0.6.0/24	Azure Bastion (rename to AzureBastionSubnet)	Microsoft required rules
snet-gateway	10.0.7.0/24	VPN or ExpressRoute Gateway	NO NSG — blocks gateway routing
snet-aks	10.0.8.0/24	AKS node pools (use /22 for large)	AKS manages its own NSG

4.3 NSG Rules — Portal Walkthrough

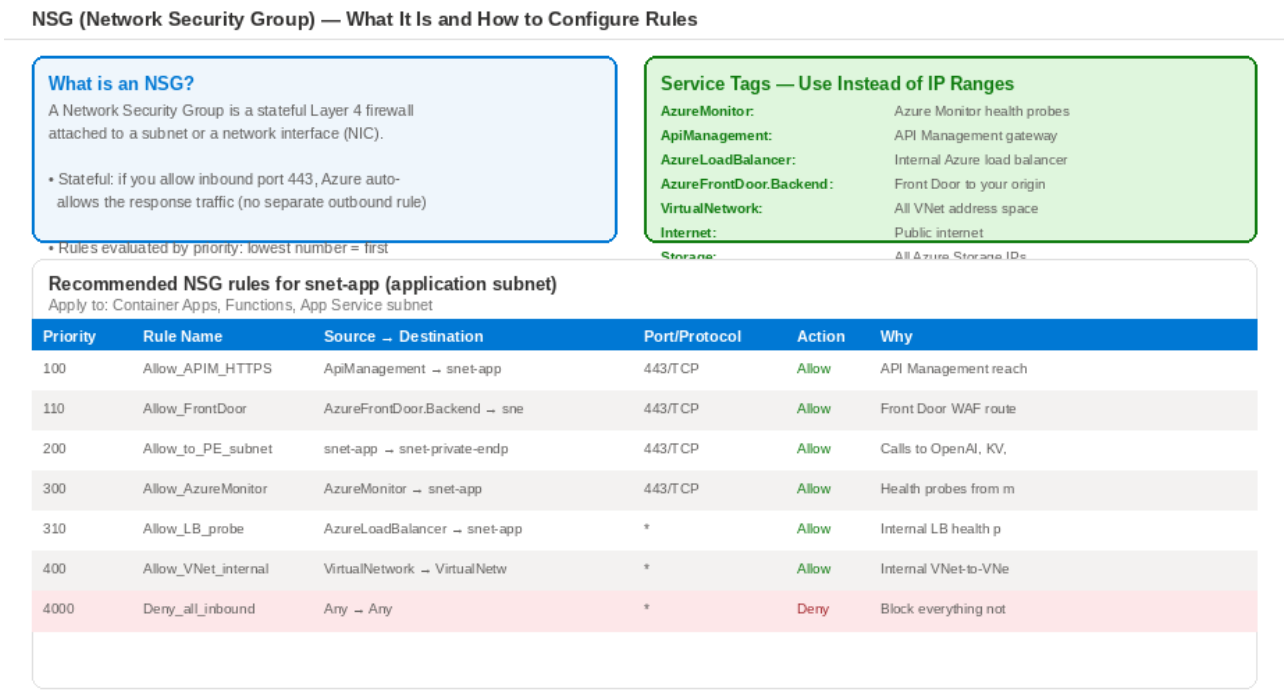


Fig 4.3 — NSG concepts: what it is, service tags, recommended rules for snet-app

1	**Create NSG** Portal → Network security groups → + Create → name: nsg-app-aiml-prod → Create
2	**Associate with subnet** NSG → Subnets → Associate → vnet-aiml-prod → snet-app → OK
3	**Add inbound rule** NSG → Inbound security rules → + Add → fill all fields → Add

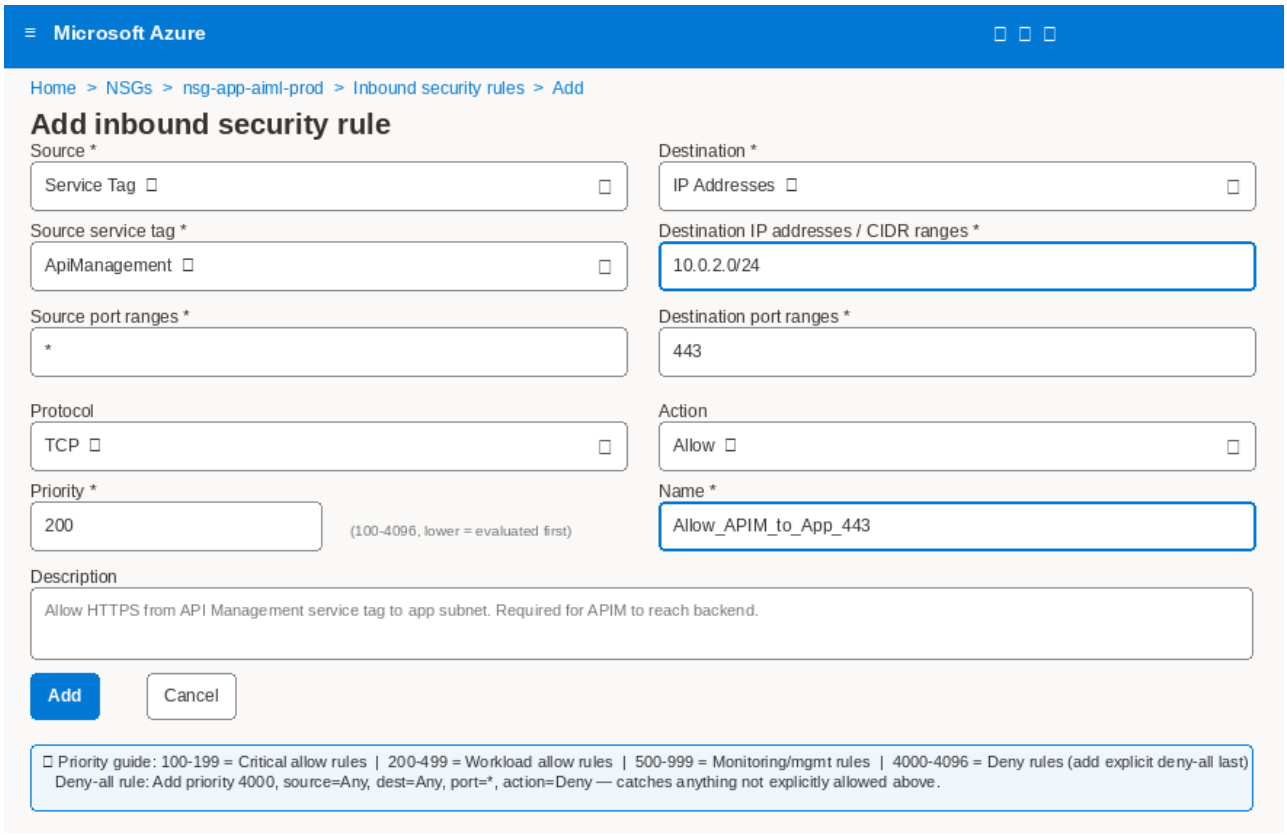


Fig 4.4 — NSG Add inbound rule form: source service tag, destination CIDR, priority guide

⚠️ Always add a Deny-all rule at priority 4000 (the highest number in your custom rules). This makes your allow-list explicit — anything not explicitly allowed is blocked. The default Azure deny-all is at 65500 and cannot be deleted, but adding one at 4000 makes your intent clear.

4.4 Private Endpoints — Complete Walkthrough

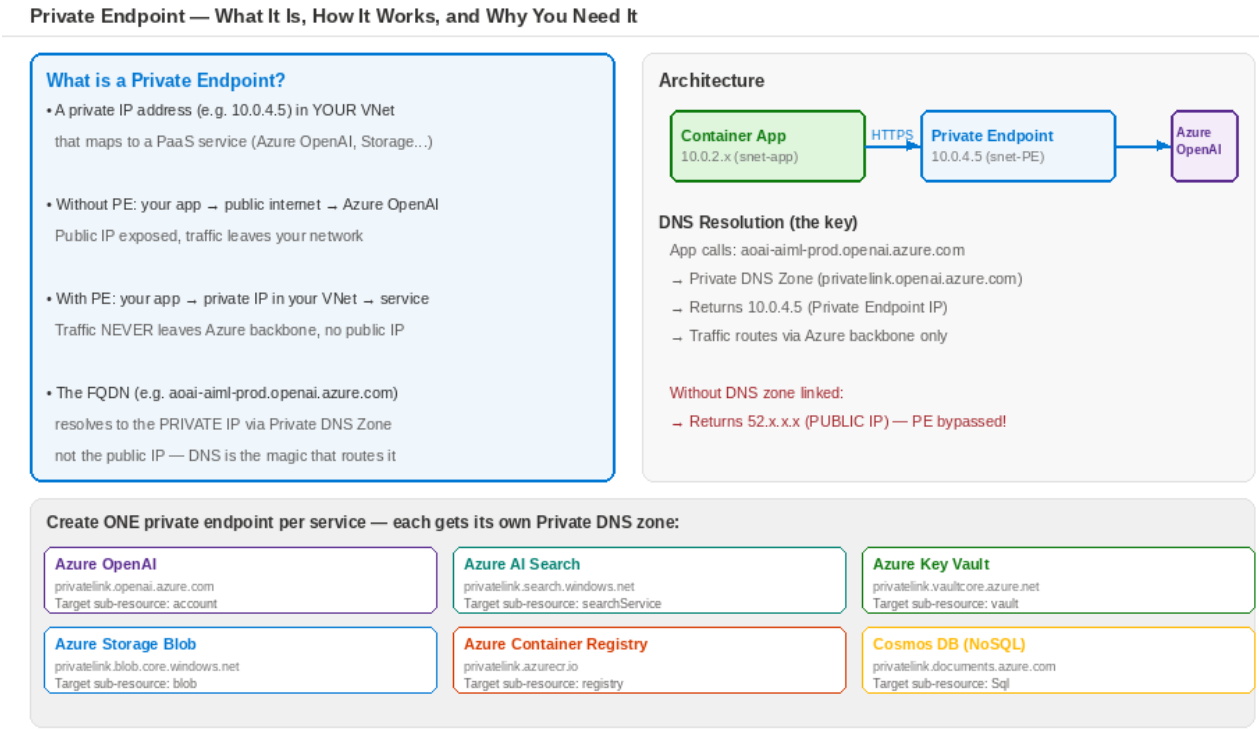


Fig 4.5 — What private endpoints are, how DNS resolution works, all service DNS zones

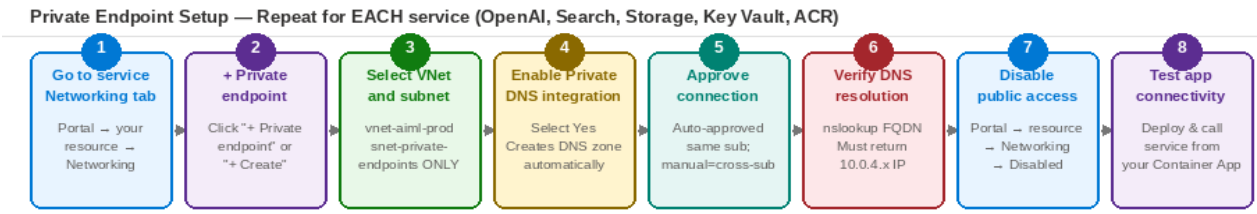


Fig 4.6 — Private endpoint setup workflow: 8 steps, repeat for each service

1 — Open service Networking tab

Portal → your OpenAI resource → Networking → Private endpoint connections → + Private endpoint

PE creation field	Enter this	Why
Name	pe-aoai-aiml-prod	Convention: pe-{service}-{project}-{env}
Subnet	snet-private-endpoints	ALWAYS use dedicated PE subnet — not snet-app
Target sub-resource	account (auto-selected)	Maps to the service endpoint you want private
Private DNS integration	Yes	Creates privatelink.openai.azure.com zone and A record automatically

2 — Verify DNS zone

Portal → Private DNS zones → privatelink.openai.azure.com → Virtual network links → confirm VNet listed

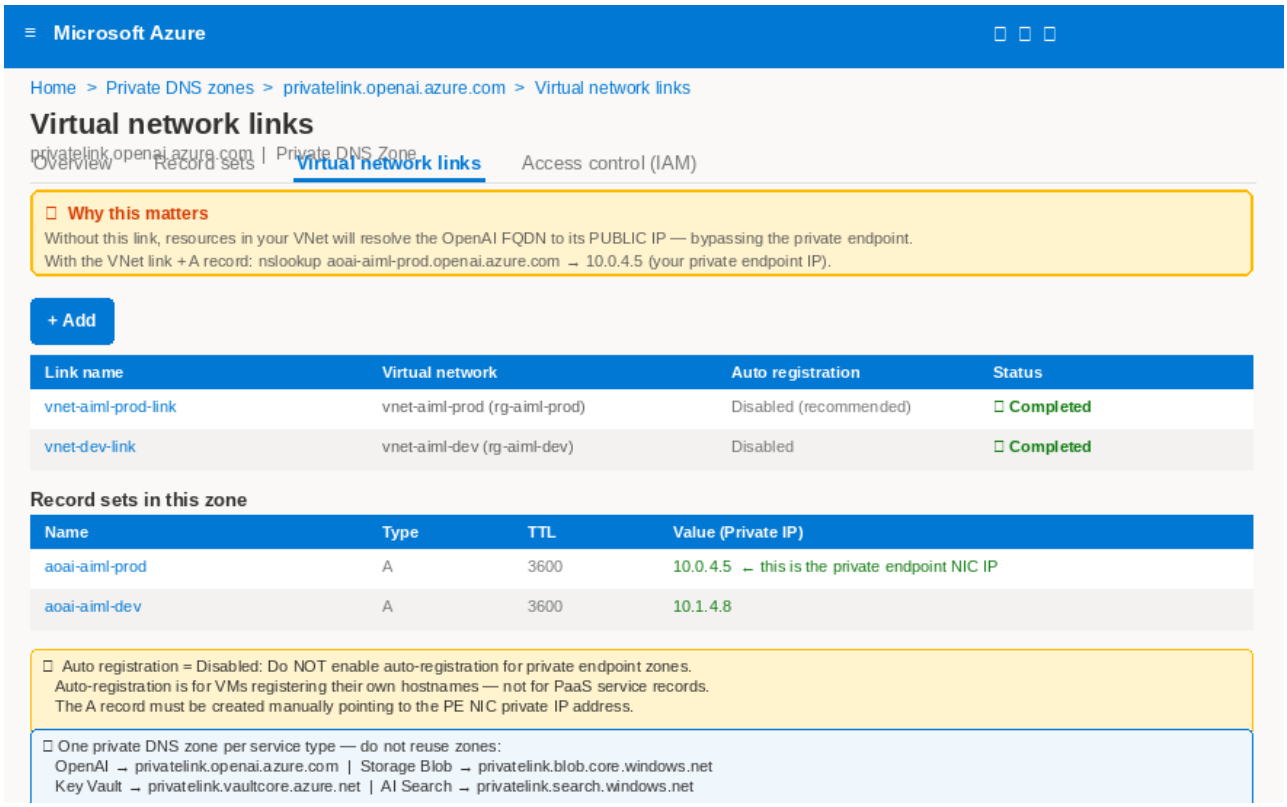


Fig 4.7 — Private DNS Zone: VNet link and A record pointing to private endpoint IP

3 **Test from inside VNet**

Open Cloud Shell → nslookup aoai-aiml-prod.openai.azure.com → must return 10.0.4.x, NOT a public IP

```
# Test private endpoint from Cloud Shell
nslookup aoai-aiml-prod.openai.azure.com

# CORRECT output (PE working):
# Name:    aoai-aiml-prod.openai.azure.com
# Address: 10.0.4.5  ← private IP from snet-private-endpoints

# WRONG output (PE NOT working — DNS zone not linked):
# Name:    aoai-aiml-prod.openai.azure.com
# Address: 52.xxx.xxx.xxx  ← public IP, traffic goes over internet!

# If you see public IP, check:
# 1. Private DNS zone privatelink.openai.azure.com exists
# 2. VNet link to vnet-aiml-prod exists in that DNS zone
# 3. A record in DNS zone matches PE NIC private IP

# Get PE NIC IP:
az network nic show --ids \
    $(az network private-endpoint show -n pe-aoai-aiml-prod -g rg-aiml-prod \
      --query networkInterfaces[0].id -o tsv) \
    --query ipConfigurations[0].privateIPAddress -o tsv
```

4 **Disable public access**

Portal → OpenAI resource → Networking → select "Disabled" → Save. Repeat for each service.

PART 4: AZURE SERVICES — SETUP, CONFIGURE & USE

Chapter 5: Azure Key Vault — Zero-Secret Pattern

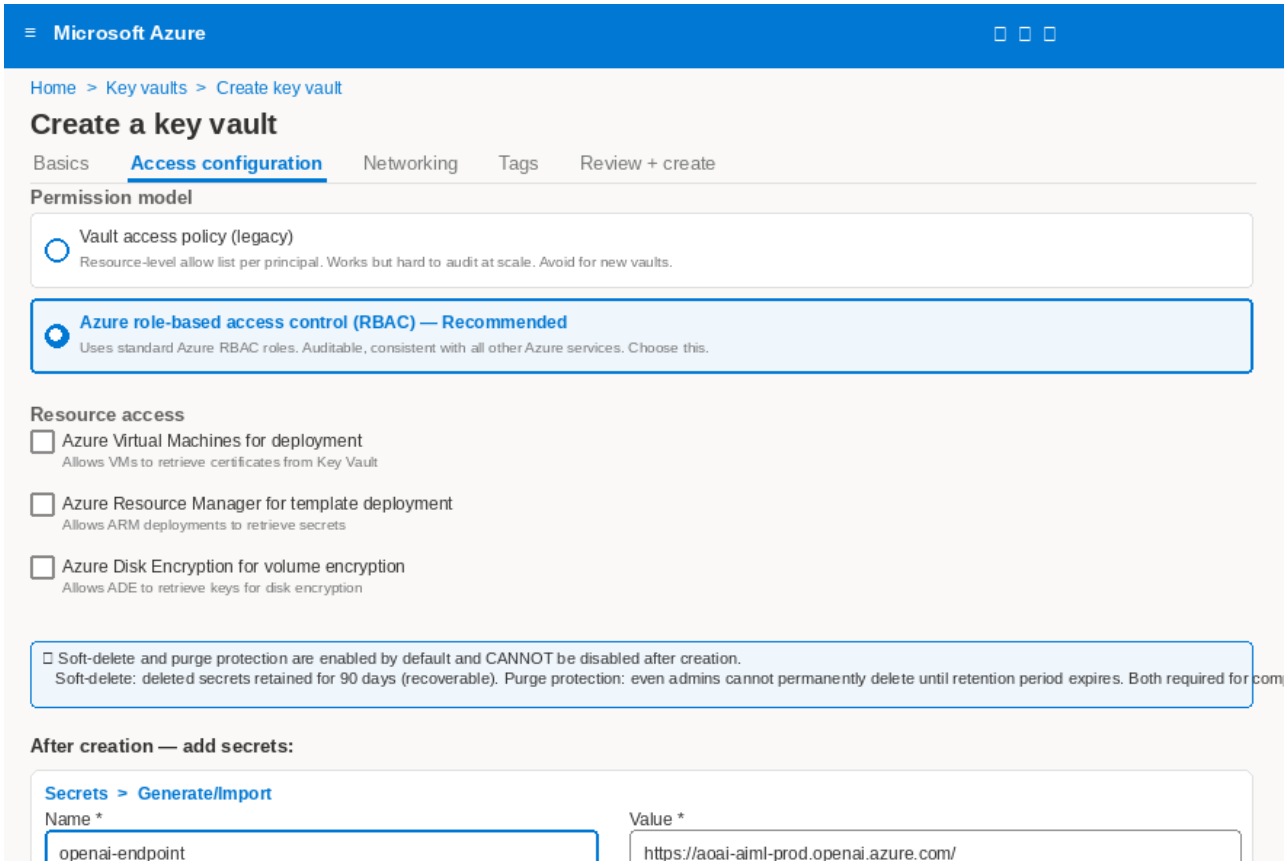


Fig 5.1 — Key Vault Access Configuration: RBAC model is required, never legacy policy

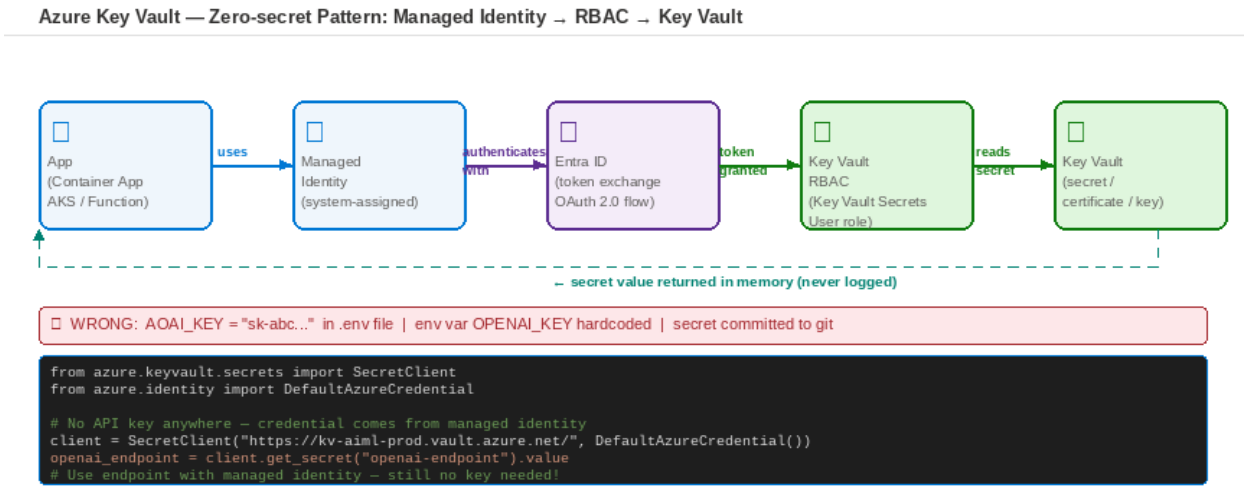


Fig 5.2 — Zero-secret pattern: app MI → KV secret read → value never in code

```
# Create KV with RBAC model, add secrets, use KV references in apps
az keyvault create -n kv-aiml-prod -g rg-aiml-prod \
    --enable-rbac-authorization --enable-purge-protection
```

```
# Store secrets
az keyvault secret set --vault-name kv-aiml-prod \
    --name "openai-endpoint" --value "https://aoai-aiml-prod.openai.azure.com/"

# Container App uses KV references — app reads as normal env var
az containerapp update -n rag-api -g rg-aiml-prod --set-env-vars \
    OPENAI_ENDPOINT="@Microsoft.KeyVault(SecretUri=https://kv-aiml-prod.vault.azure.net/secrets/
openai-endpoint/)"

# Python: read directly when you need dynamic config
from azure.keyvault.secrets import SecretClient
from azure.identity import DefaultAzureCredential
kv = SecretClient("https://kv-aiml-prod.vault.azure.net/", DefaultAzureCredential())
value = kv.get_secret("openai-endpoint").value
```

Chapter 6: Azure Container Registry (ACR)

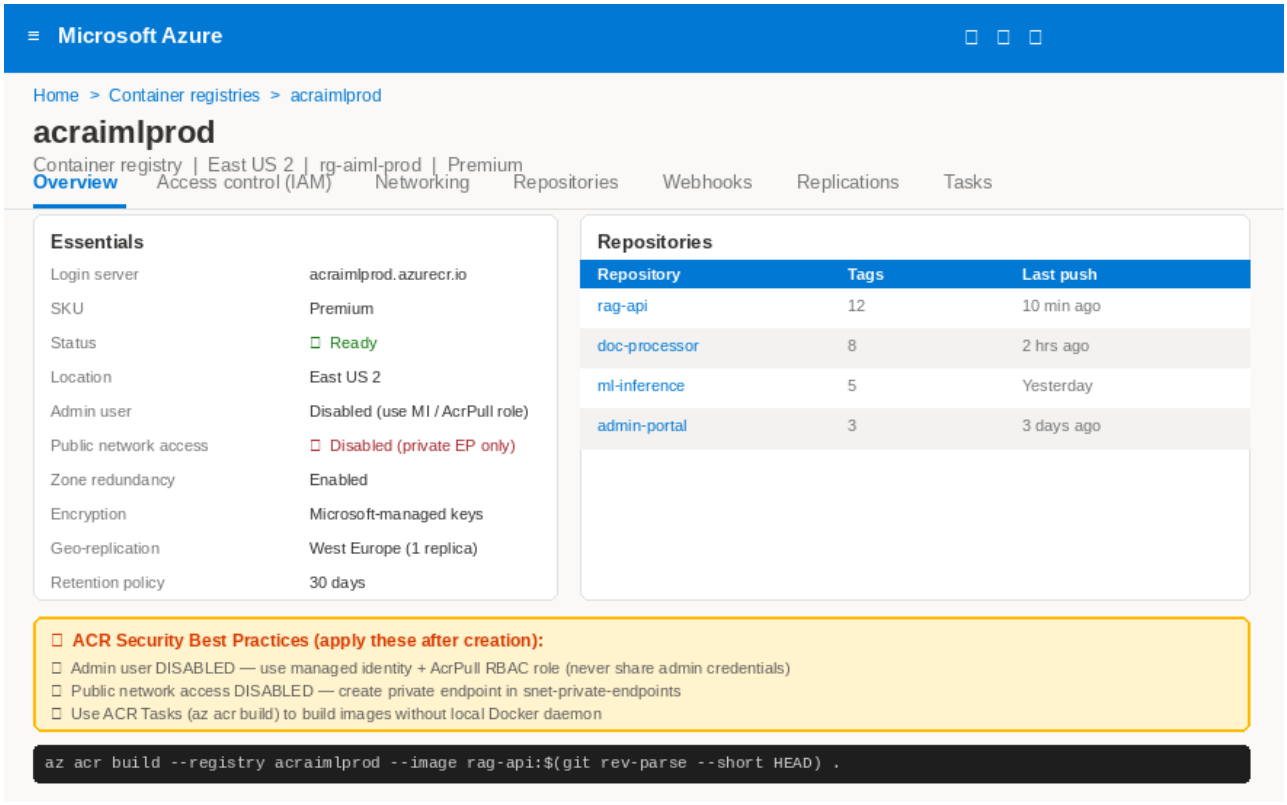


Fig 6.1 — ACR overview: login server, SKU (Premium), geo-replication, security settings

```
# Create Premium ACR with private endpoint
az acr create -n acraimlprod -g rg-aiml-prod --sku Premium --admin-enabled false

# Build image in ACR (no local Docker needed)
az acr build --registry acraimlprod \
    --image rag-api:$(git rev-parse --short HEAD) \
    --image rag-api:latest  .

# Grant Container App MI the AcrPull role
az role assignment create --role AcrPull \
    --assignee $(az containerapp show -n rag-api -g rg-aiml-prod --query identity.principalId -o
tsv) \
    --scope $(az acr show -n acraimlprod -g rg-aiml-prod --query id -o tsv)
```

Chapter 7: Container Apps Environment & Container Apps

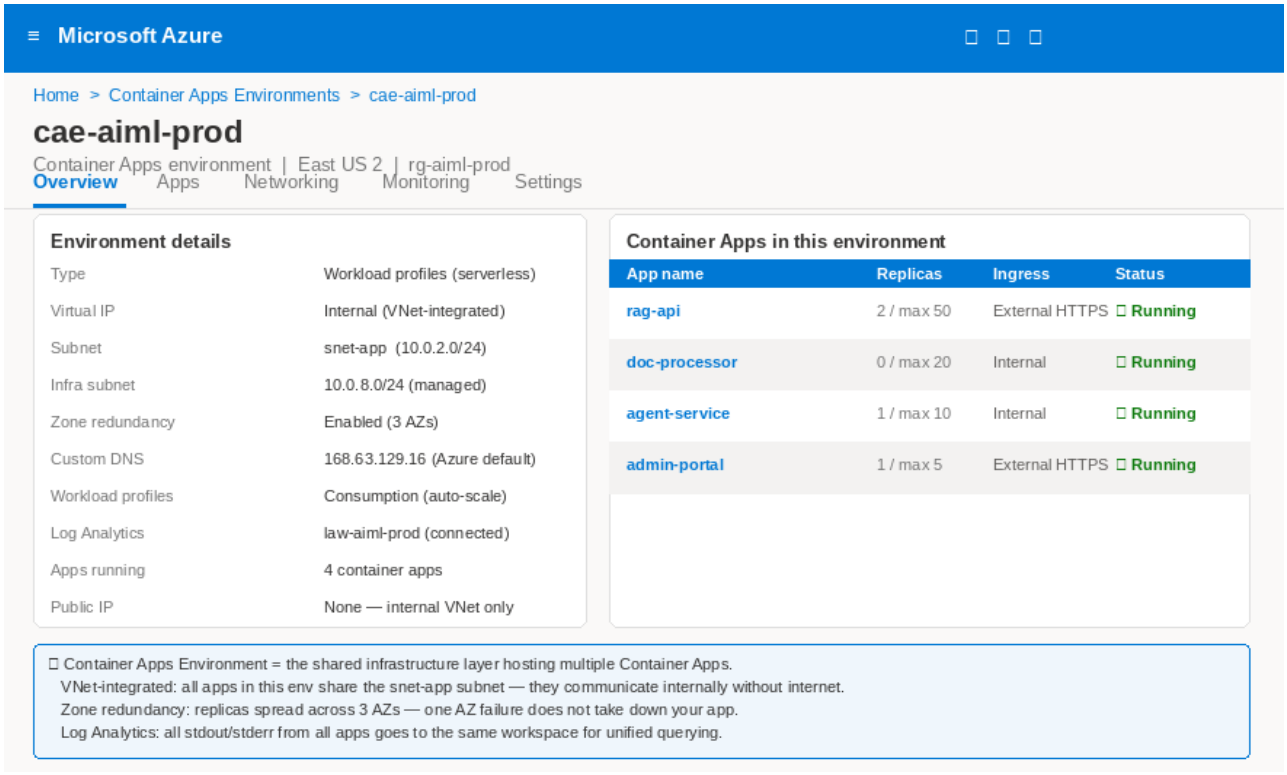


Fig 7.1 — Container Apps Environment: VNet-integrated, zone-redundant, shared infrastructure

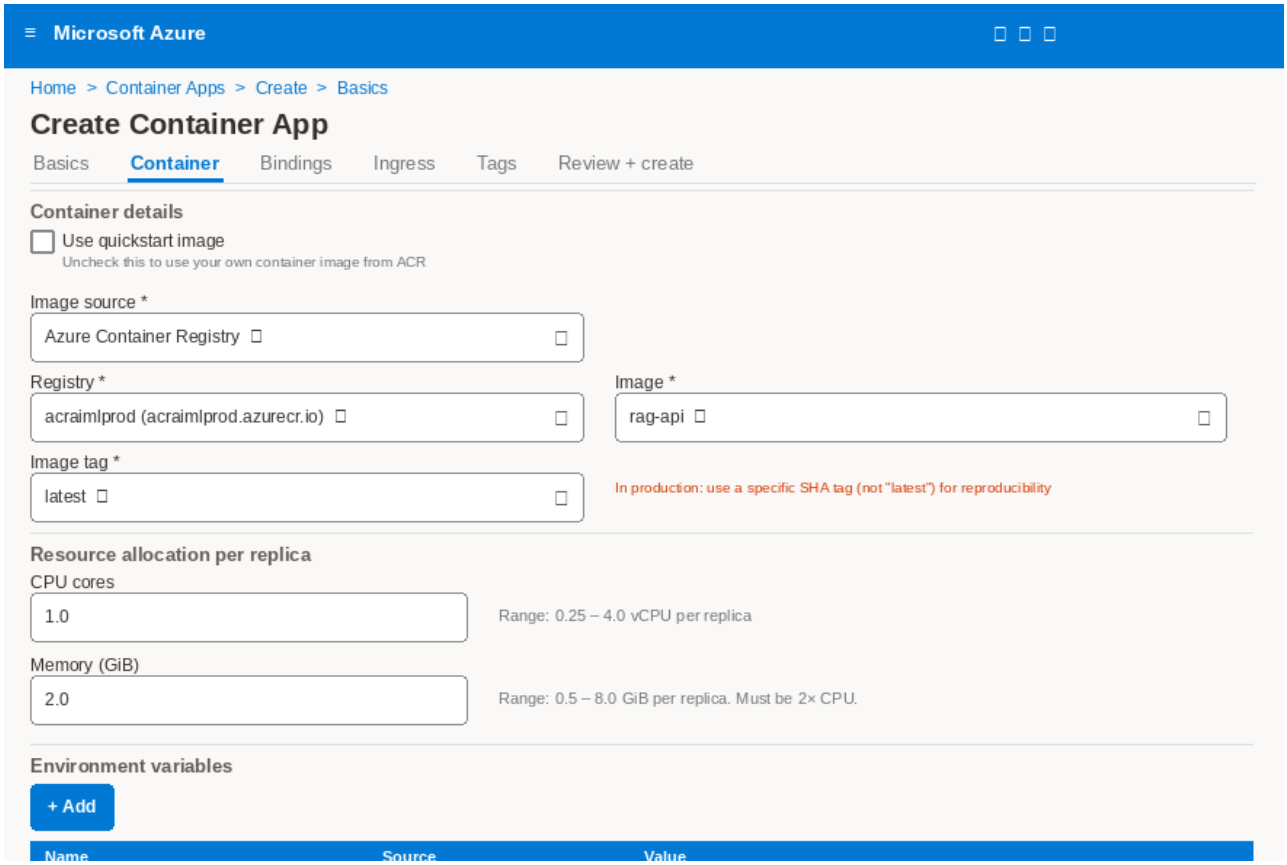


Fig 7.2 — Container App creation: image source, resource allocation, Key Vault secret refs

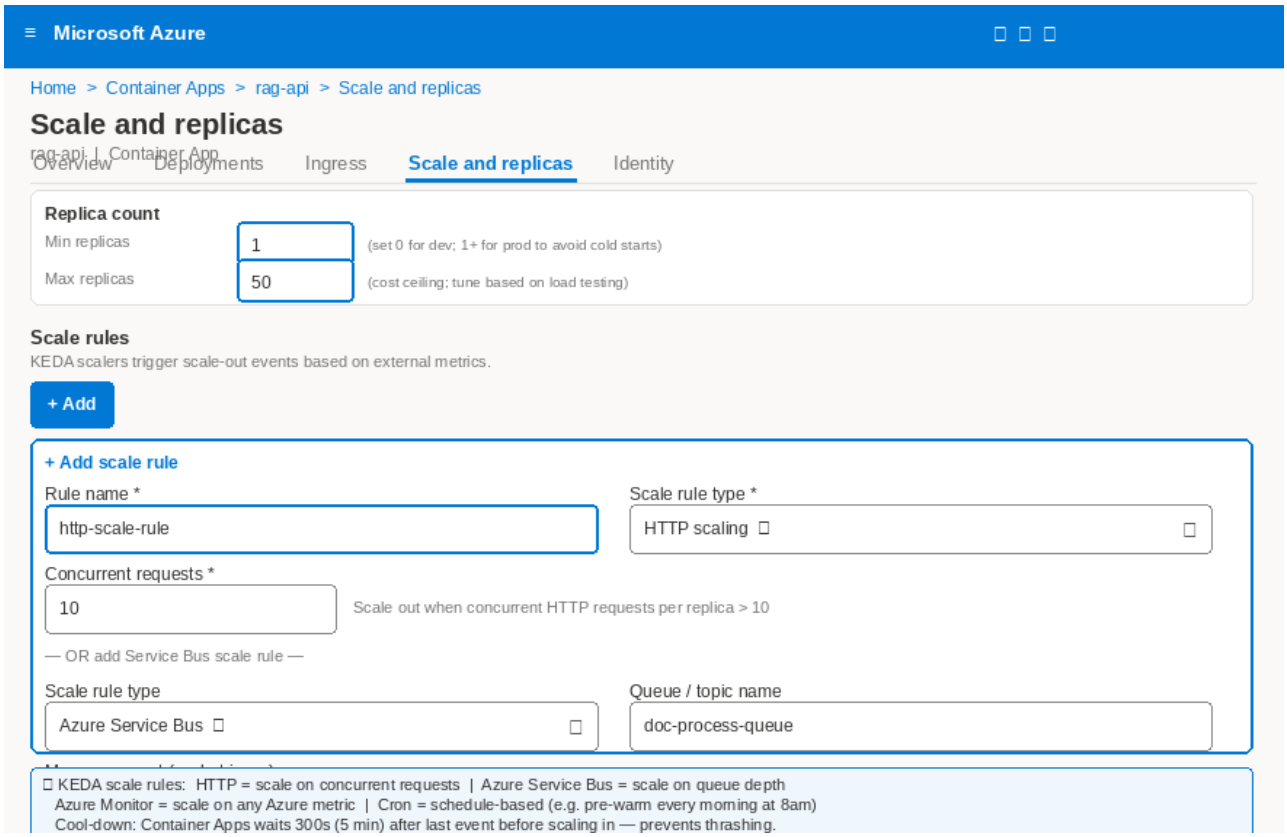


Fig 7.3 — Scale rules: min/max replicas, HTTP KEDA scaler, Service Bus queue scaler

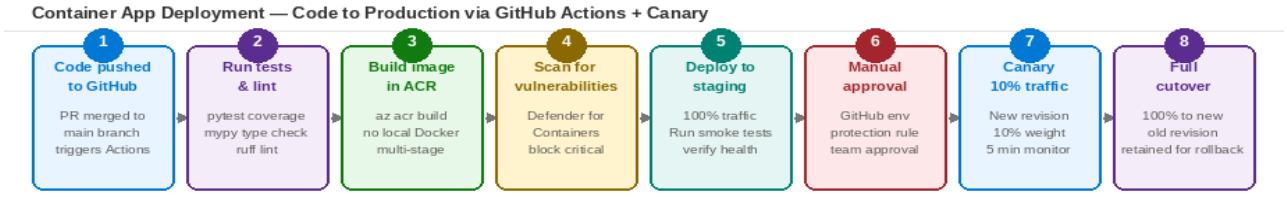


Fig 7.4 — Deployment workflow: code → ACR → staging → manual approval → canary → production

```
# Create CAE with VNet integration
az containerapp env create -n cae-aiml-prod -g rg-aiml-prod \
    --location eastus2 \
    --infrastructure-subnet-resource-id $(az network vnet subnet show \
        -n snet-app --vnet-name vnet-aiml-prod -g rg-aiml-prod --query id -o tsv)

# Create Container App with MI, KV refs, KEDA scale rule
az containerapp create -n rag-api -g rg-aiml-prod \
    --environment cae-aiml-prod \
    --image acraimlprod.azurecr.io/rag-api:latest \
    --target-port 8000 --ingress external \
    --min-replicas 1 --max-replicas 50 \
    --scale-rule-name http-rule --scale-rule-http-concurrency 10 \
    --cpu 1.0 --memory 2.0Gi \
    --system-assigned \
    --set-env-vars ENVIRONMENT=production \
    OPENAI_ENDPOINT="@Microsoft.KeyVault(SecretUri=https://kv-aiml-prod.vault.azure.net/secrets/
openai-endpoint/)"
```

Chapter 8: App Service & Azure Functions

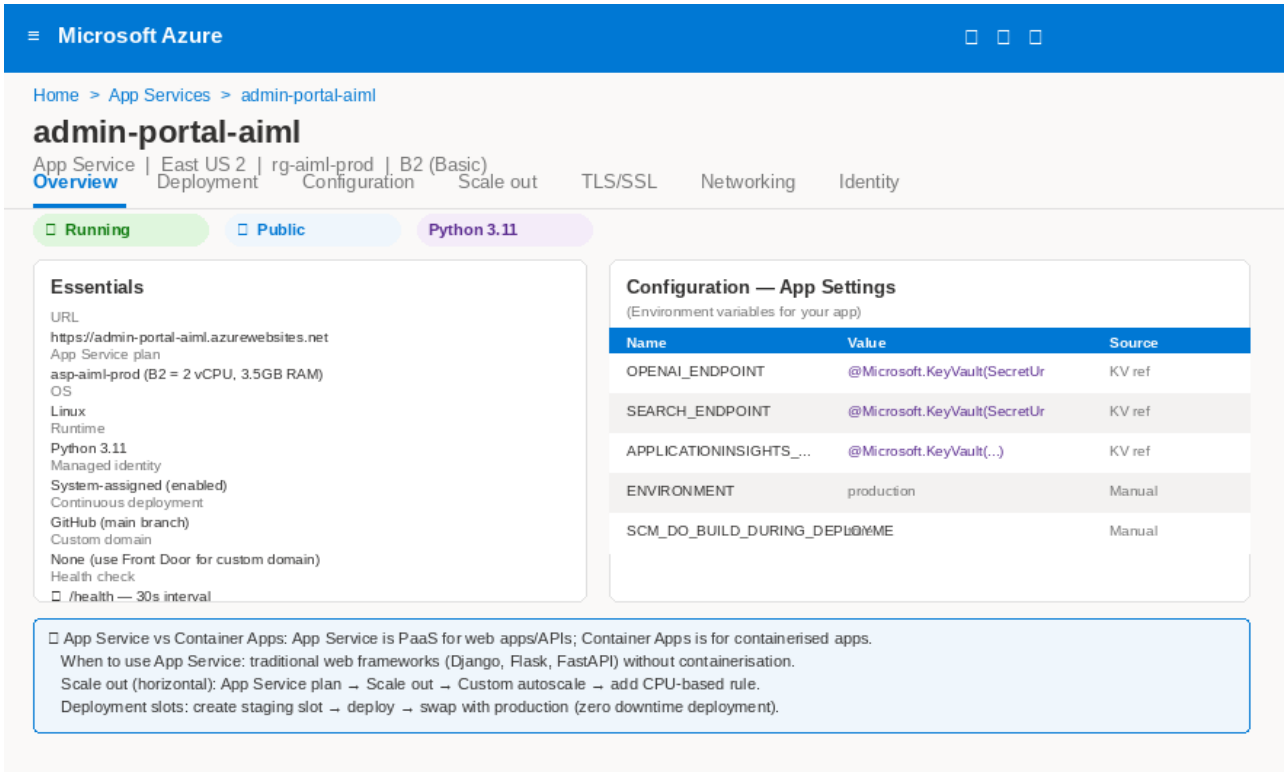


Fig 8.1 — App Service: URL, plan, runtime, MI, deployment settings, app configuration

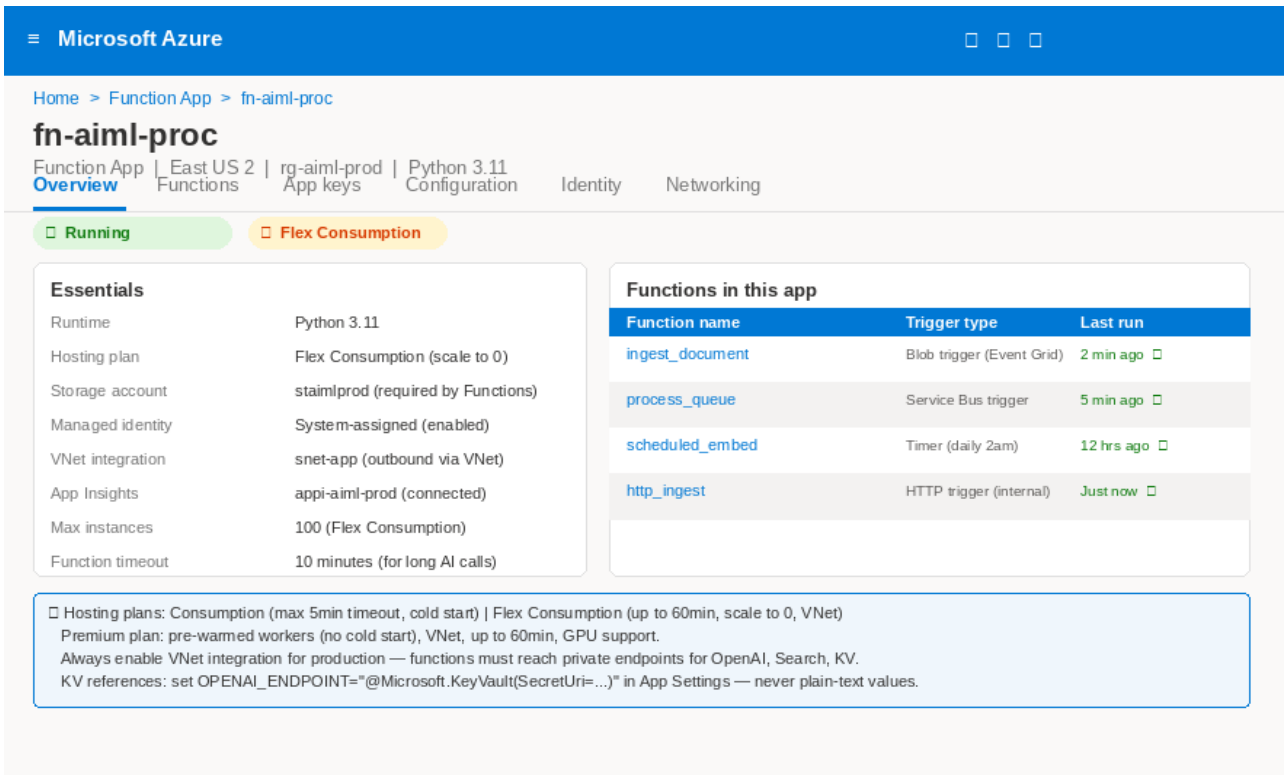


Fig 8.2 — Function App: functions list, trigger types, hosting plan, VNet integration

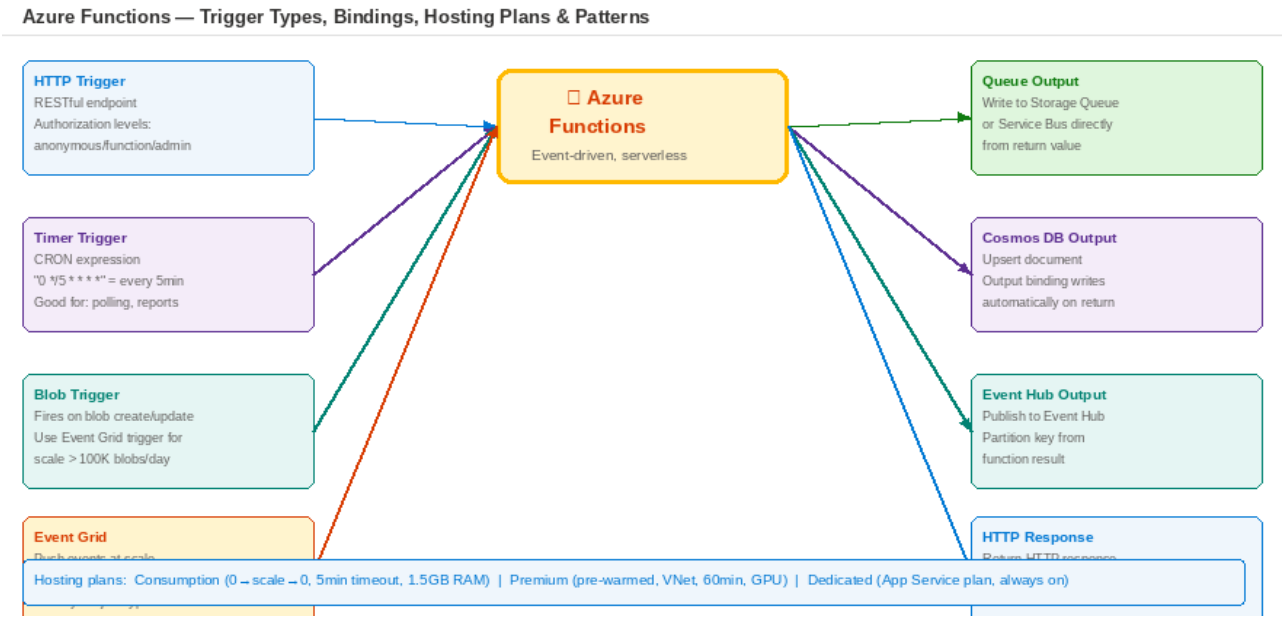


Fig 8.3 — Function trigger types: HTTP, Timer, Blob, Event Grid, Service Bus — all shown

```
# App Service — Python web app
az appservice plan create -n asp-aiml-prod -g rg-aiml-prod --sku B2 --is-linux
az webapp create -n admin-portal-aiml -g rg-aiml-prod -p asp-aiml-prod --runtime "PYTHON:3.11"
az webapp identity assign -n admin-portal-aiml -g rg-aiml-prod

# Azure Functions — Flex Consumption (scale to 0, VNet access)
az functionapp create -n fn-aiml-proc -g rg-aiml-prod \
    --storage-account staimlprod \
    --flexconsumption-location eastus2 \
    --runtime python --runtime-version 3.11 \
    --assign-identity "[system]"

# Function triggered by new blob in Storage — ingestion pipeline
import azure.functions as func
app = func.FunctionApp()

@app.blob_trigger(arg_name="blob", path="raw-docs/{name}",
                  connection="AzureWebJobsStorage")
def ingest_document(blob: func.InputStream) -> None:
    # extract text → chunk → embed → upsert to AI Search
    pass
```

Chapter 9: Storage Account — Blob, ADLS Gen2 & Lifecycle

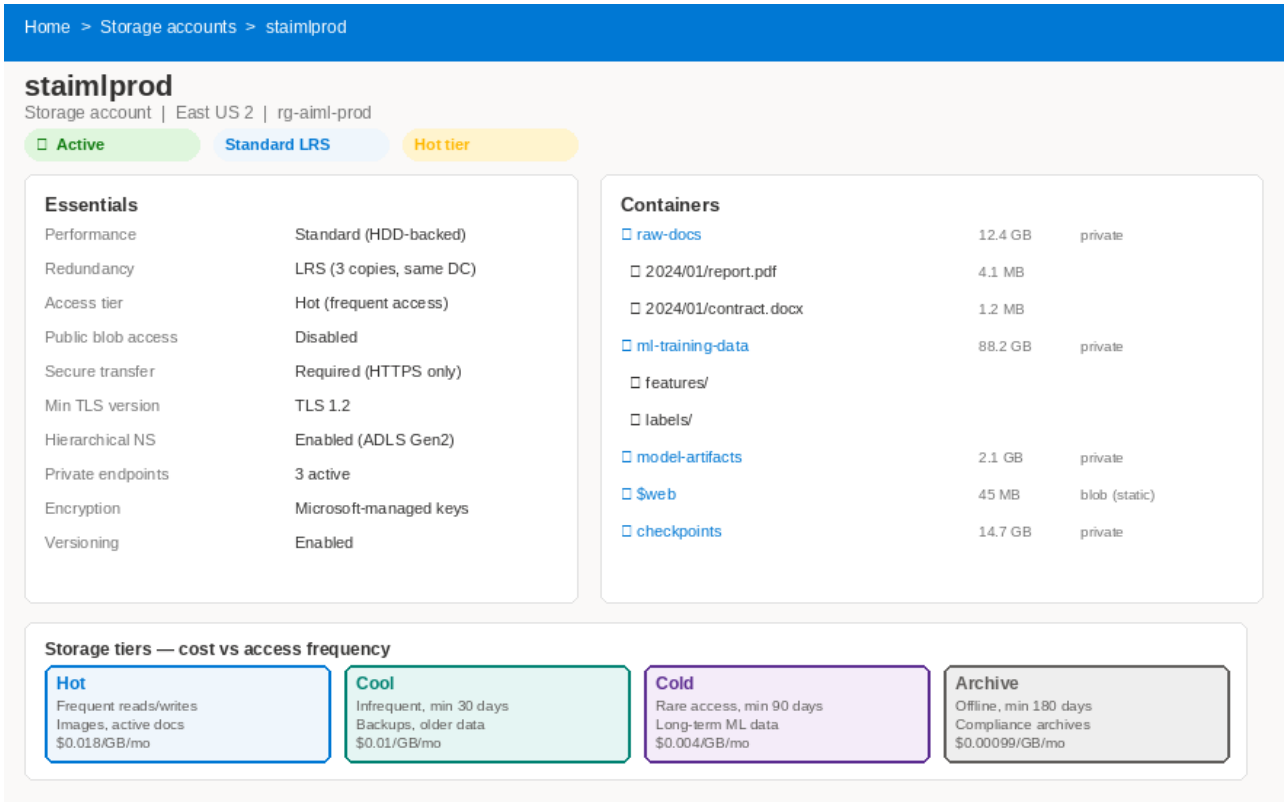


Fig 9.1 — Storage account: properties, container hierarchy, access tier comparison

```
# Create with ADLS Gen2 (hierarchical namespace) + security hardening
az storage account create -n staimlprod -g rg-aiml-prod -l eastus2 \
    --sku Standard_ZRS --kind StorageV2 \
    --enable-hierarchical-namespace true \
    --allow-blob-public-access false --min-tls-version TLS1_2 \
    --public-network-access Disabled

# Python SDK — upload, download, list (keyless auth)
from azure.storage.blob import BlobServiceClient
from azure.identity import DefaultAzureCredential

client = BlobServiceClient("https://staimlprod.blob.core.windows.net",
                           DefaultAzureCredential())

blob = client.get_blob_client("raw-docs", "2024/q4-report.pdf")
with open("q4.pdf", "rb") as f:
    blob.upload_blob(f, overwrite=True)

data = blob.download_blob().readall()

for b in client.get_container_client("raw-docs").list_blobs(name_starts_with="2024/"):
    print(f"{b.name}: {b.size:,} bytes, modified: {b.last_modified}")
```

Chapter 10: API Management

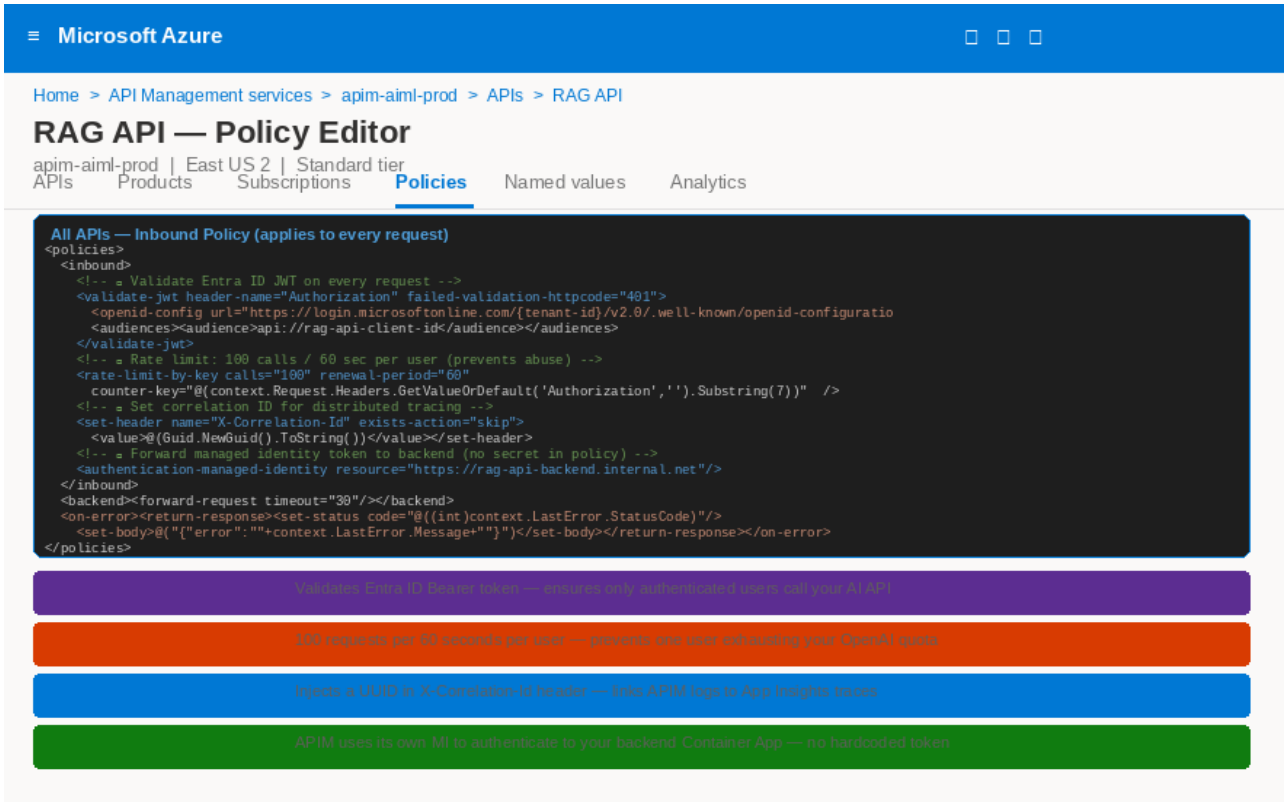


Fig 10.1 — APIM policy editor: JWT validation, rate limiting, correlation ID, MI forwarding

```
# APIM all-APIs inbound policy — paste in portal: APIM → APIs → All operations → Policy
<policies>
  <inbound>
    <!-- 1: Validate Entra ID JWT -->
    <validate-jwt header-name="Authorization" failed-validation-httpcode="401">
      <openid-config url="https://login.microsoftonline.com/{tenant}/v2.0/.well-known/openid-
configuration"/>
      <audiences><audience>api://rag-api-client-id</audience></audiences>
    </validate-jwt>
    <!-- 2: Rate limit 100/min per user -->
    <rate-limit-by-key calls="100" renewal-period="60"
      counter-key="@(context.Request.Headers['Authorization'].Last().Substring(7))"/>
    <!-- 3: Correlation ID for distributed tracing -->
    <set-header name="X-Correlation-Id" exists-action="skip">
      <value>@(Guid.NewGuid().ToString())</value></set-header>
  </inbound>
  <backend><forward-request timeout="30"/></backend>
</policies>
```

Chapter 11: Azure AI Search

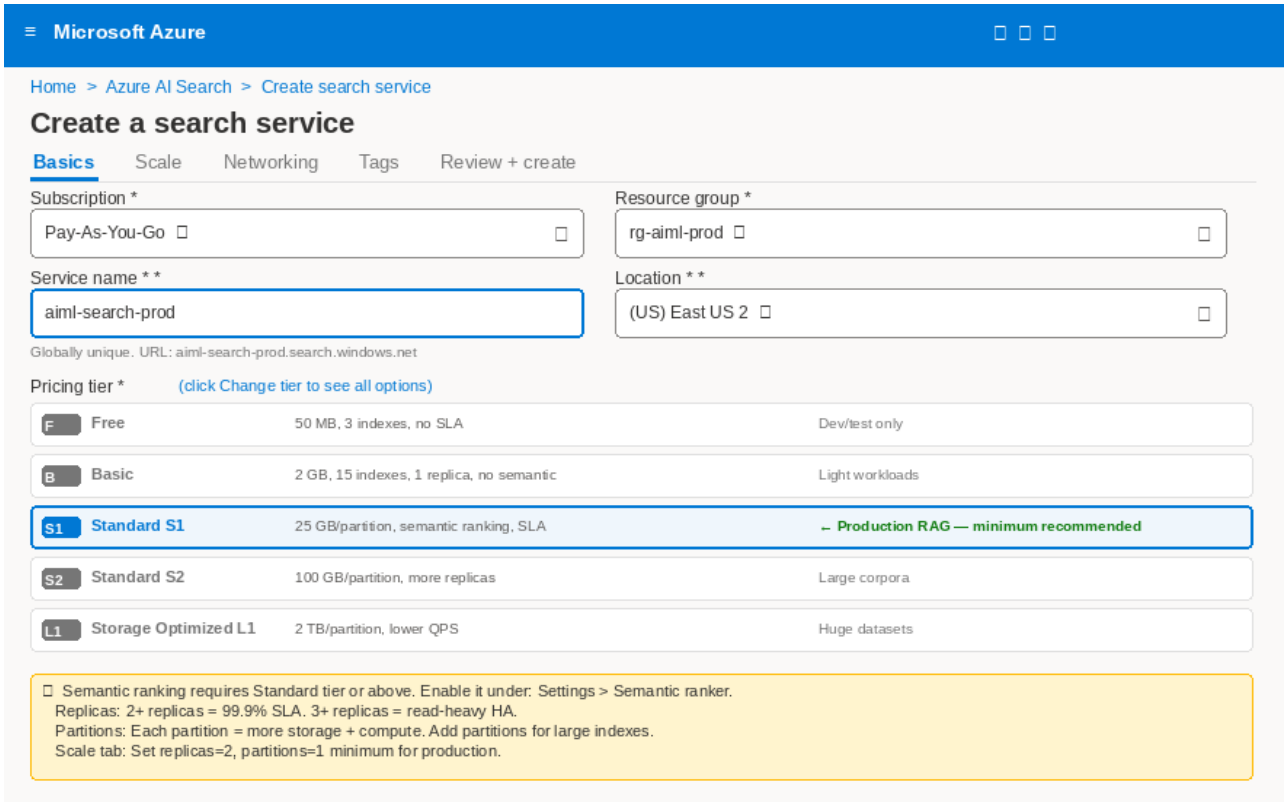


Fig 11.1 — AI Search creation: tier comparison (S1 minimum), replicas for SLA

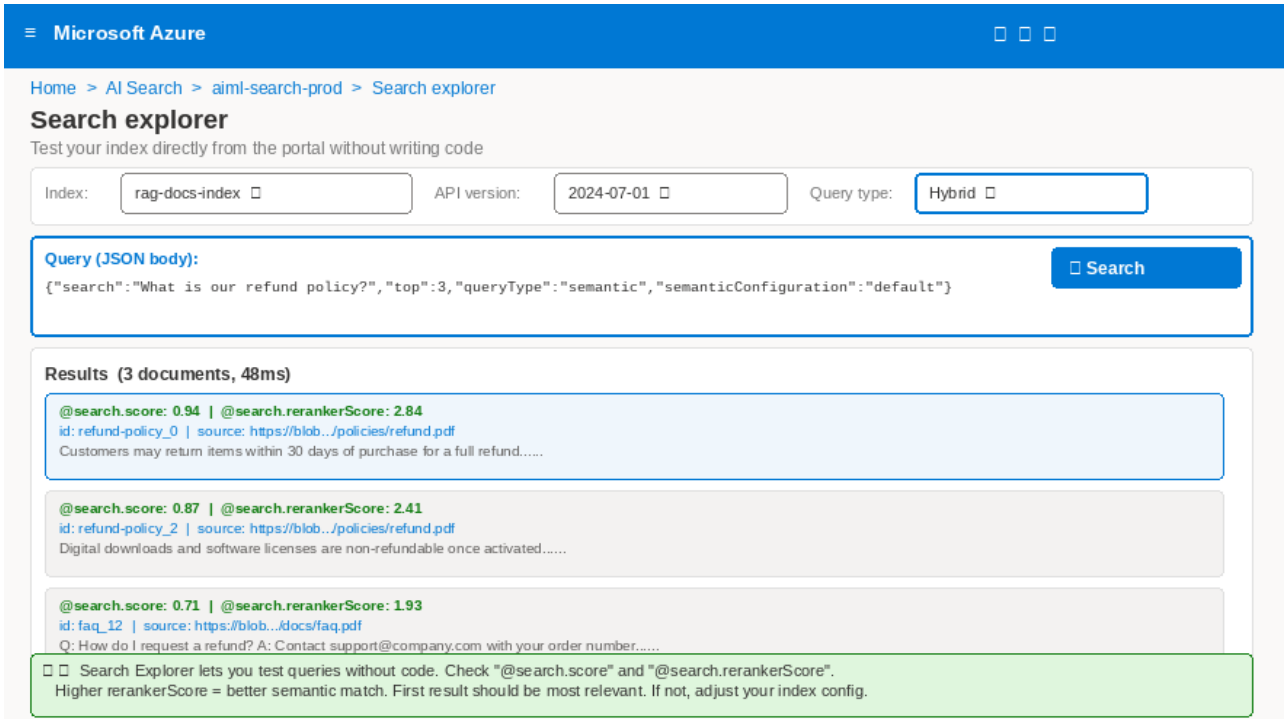


Fig 11.2 — Search Explorer: test hybrid queries, relevance scores, validate index

```
# Create index + hybrid query (production RAG pattern)
from azure.search.documents.indexes import SearchIndexClient
from azure.search.documents.indexes.models import (
```

```
    SearchIndex, SimpleField, SearchableField, SearchField,
    VectorSearch, HnswAlgorithmConfiguration, VectorSearchProfile,
    SemanticConfiguration, SemanticSearch, SemanticPrioritizedFields, SemanticField)
from azure.search.documents import SearchClient
from azure.search.documents.models import VectorizedQuery

# Create index
index = SearchIndex(name="rag-docs-index", fields=[
    SimpleField("id", "Edm.String", key=True, filterable=True),
    SearchableField("content", "Edm.String", analyzer_name="en.microsoft"),
    SimpleField("source_url", "Edm.String", filterable=True, retrievable=True),
    SimpleField("tenant_id", "Edm.String", filterable=True),
    SearchField("embedding", "Collection(Edm.Single)",
                vector_search_dimensions=3072, vector_search_profile_name="hnsw"),
], vector_search=VectorSearch(
    profiles=[VectorSearchProfile("hnsw","hnsw-algo")],
    algorithms=[HnswAlgorithmConfiguration("hnsw-algo",
                parameters={"m":4,"efConstruction":400,"metric":"cosine"})]),
   semantic_search=SemanticSearch(configurations=[
    SemanticConfiguration("default", SemanticPrioritizedFields(
        content_fields=[SemanticField("content")]))]))

SearchIndexClient(ENDPOINT, cred).create_or_update_index(index)

# Hybrid search (BM25 + vector + semantic reranker)
results = SearchClient(ENDPOINT, "rag-docs-index", cred).search(
    search_text=question,
    vector_queries=[VectorizedQuery(vector=q_embedding, k_nearest_neighbors=5,
fields="embedding")],
    filter=f"tenant_id eq '{tenant_id}'",   # ALWAYS filter by tenant for data isolation
    query_type="semantic", semantic_configuration_name="default", top=5)
```

Chapter 12: Azure AI Foundry

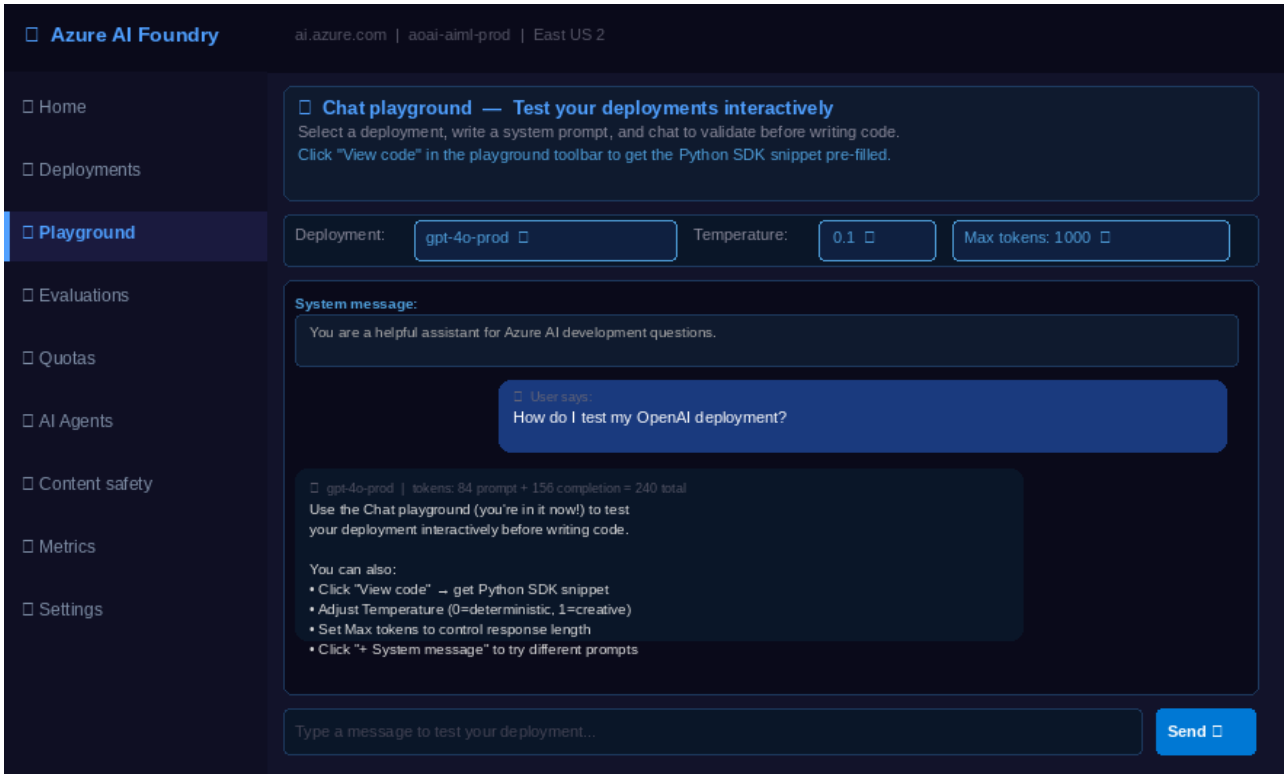


Fig 12.1 — AI Foundry playground: test deployments, see token counts, get Python code

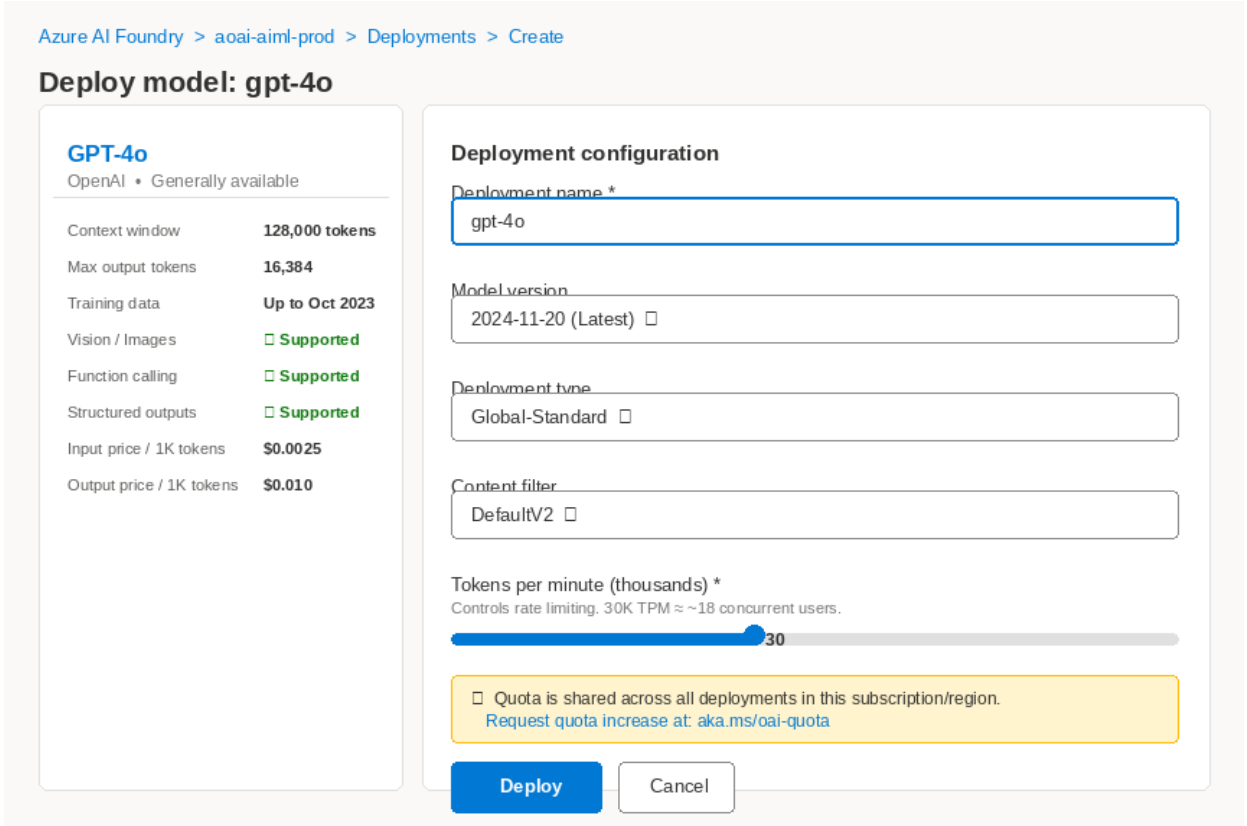


Fig 12.2 — Model deployment: Global-Standard type, content filter, TPM quota

Setting	Value	Why
Deployment name	gpt-4o-prod	Your CODE references this name, not the model name.
Model	gpt-4o	Best quality. Use gpt-4o-mini for classification (5x cheaper).
Deployment type	Global-Standard	Fewer 429 errors — uses global spare capacity.
Content filter	DefaultV2	Blocks harmful categories. Create custom profiles for specialized domains.
TPM quota	30K to start	Request increase: AI Foundry → Quotas → deployment → Request.

Chapter 13: Log Analytics Workspace & Application Insights

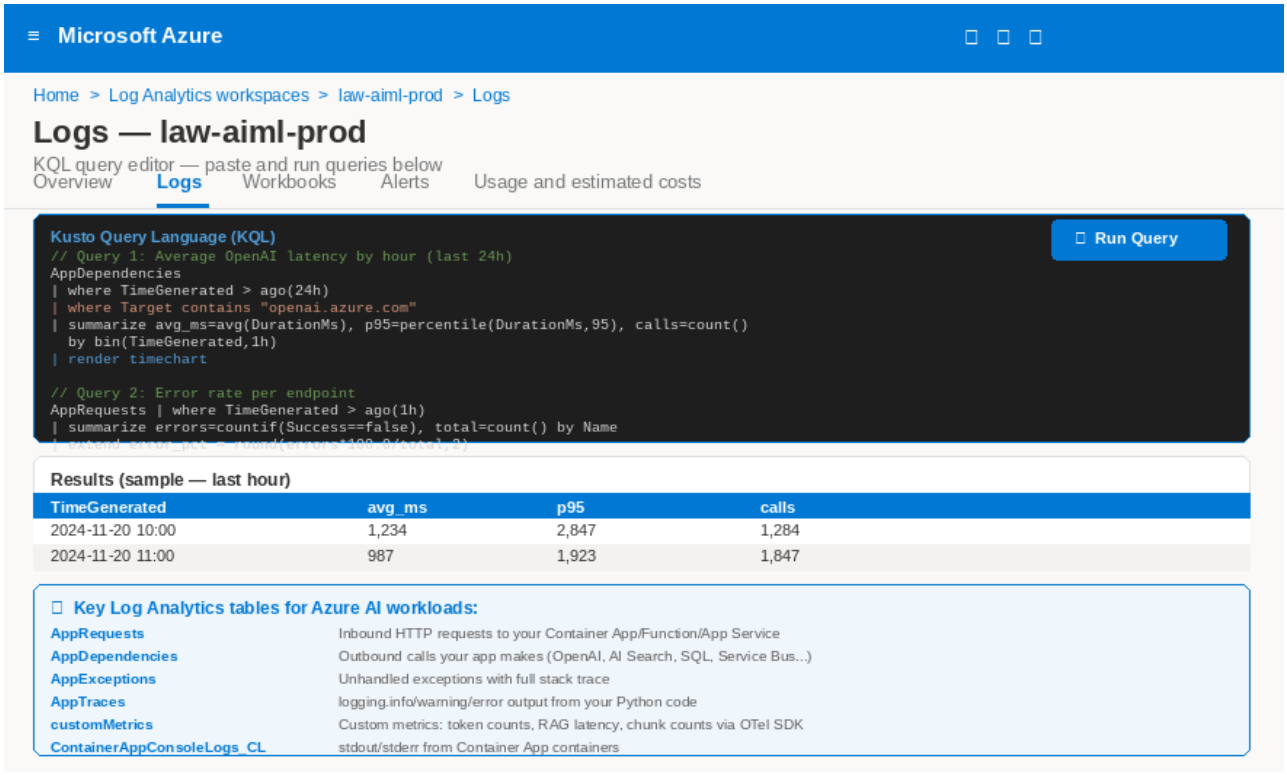


Fig 13.1 — Log Analytics: KQL editor with sample queries and table reference

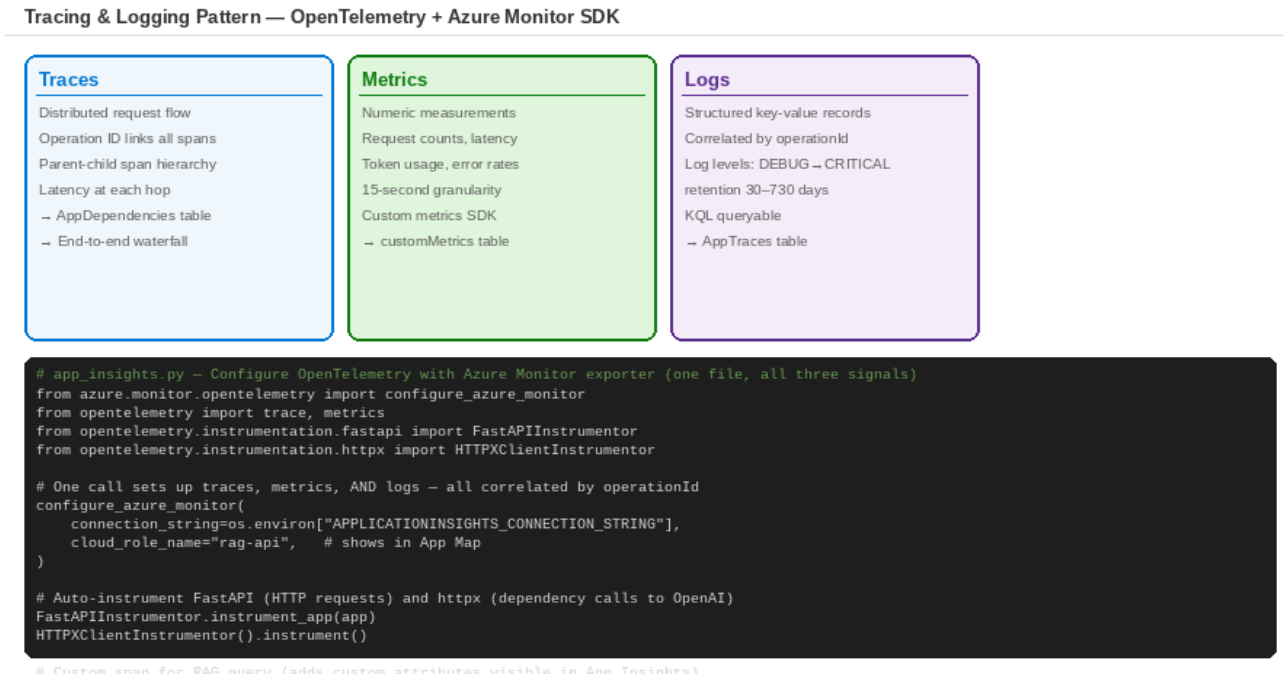


Fig 13.2 — OpenTelemetry SDK: one configure call instruments traces, metrics, logs

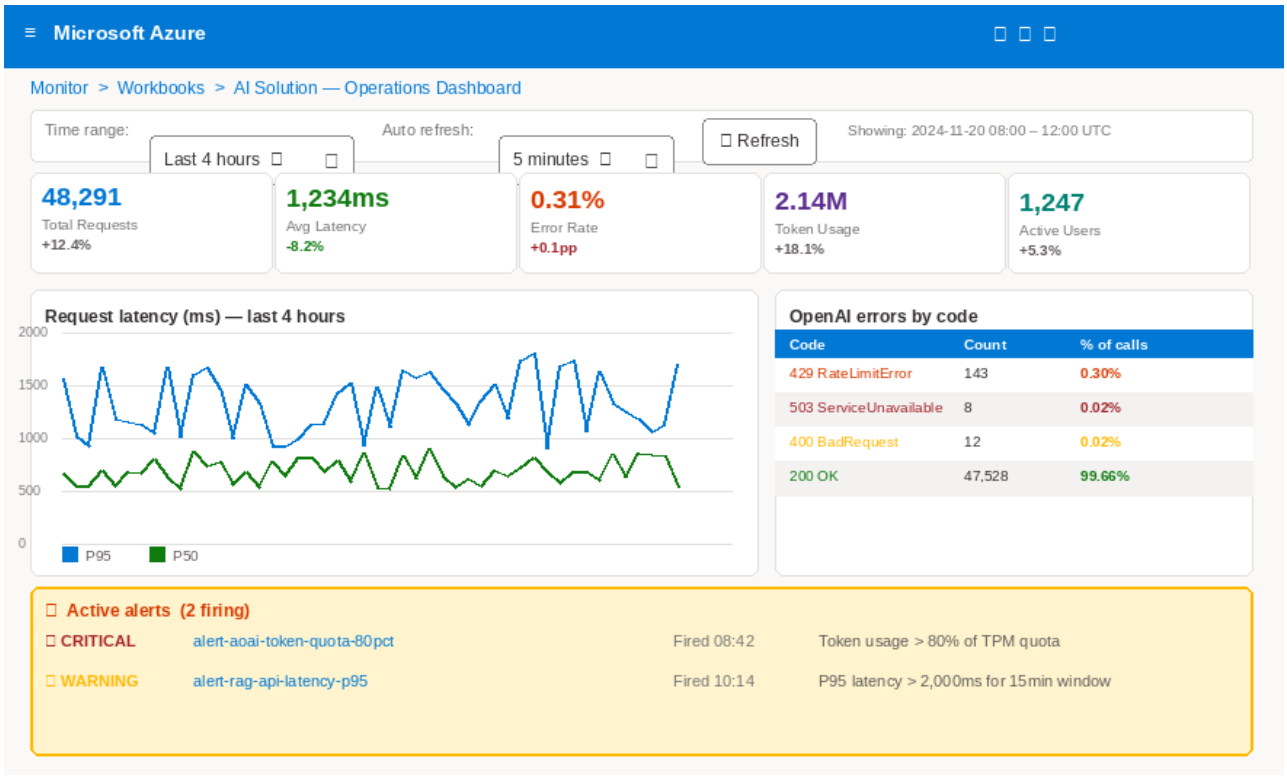


Fig 13.3 — Operations dashboard: KPI cards, latency chart, error log, active alerts

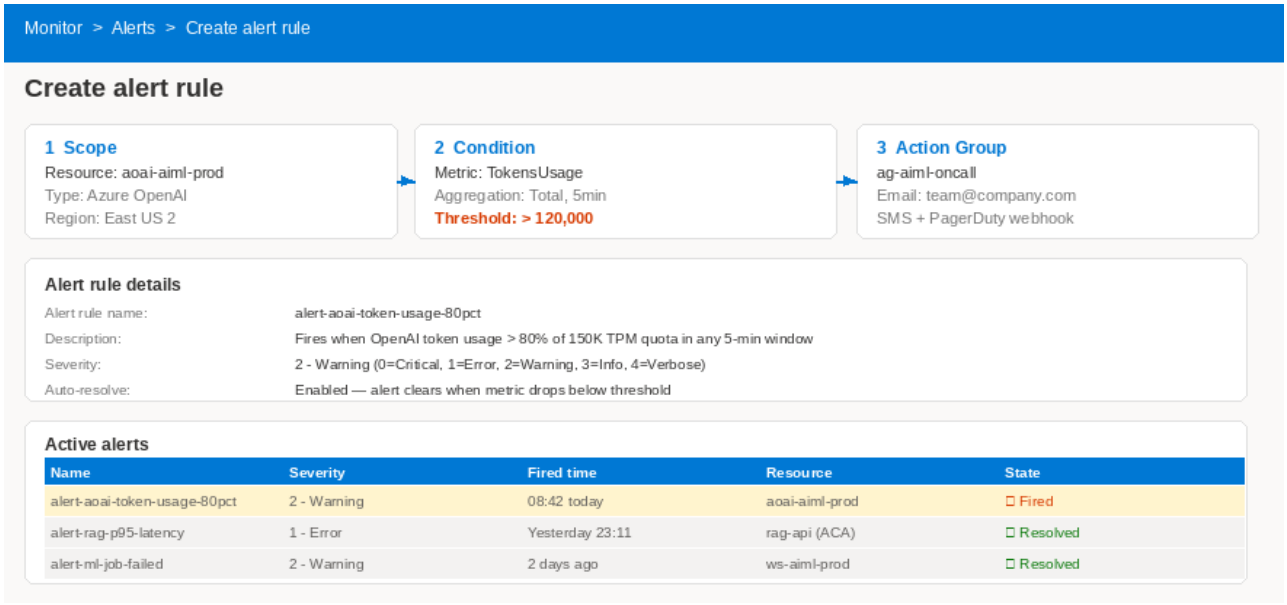


Fig 13.4 — Alert rule: scope, condition (token quota 80%), action group

```
# Setup observability — call once at startup
from azure.monitor.opentelemetry import configure_azure_monitor
from opentelemetry import trace
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.httpx import HTTPXClientInstrumentor

configure_azure_monitor(
    connection_string=os.environ["APPLICATIONINSIGHTS_CONNECTION_STRING"],
    cloud_role_name="rag-api",
)
```

```
FastAPIInstrumentor.instrument_app(app)    # auto-traces all HTTP requests
HTTPXClientInstrumentor().instrument()     # auto-traces OpenAI SDK HTTP calls

# Add custom attributes to the auto-created span
@app.post("/api/rag/query")
async def rag_query(body: QueryRequest):
    span = trace.get_current_span()
    span.set_attribute("rag.user_id", body.user_id)
    span.set_attribute("rag.chunks_retrieved", len(chunks))
    span.set_attribute("rag.prompt_tokens", usage.prompt_tokens)

# KQL: latency trend — paste in Monitor → Log Analytics → Logs
// AppDependencies | where TimeGenerated > ago(1h)
// | where Target contains "openai.azure.com"
// | summarize p50=percentile(DurationMs,50), p95=percentile(DurationMs,95)
//   by bin(TimeGenerated, 5m) | render timechart
```

Chapter 14: Cost Management

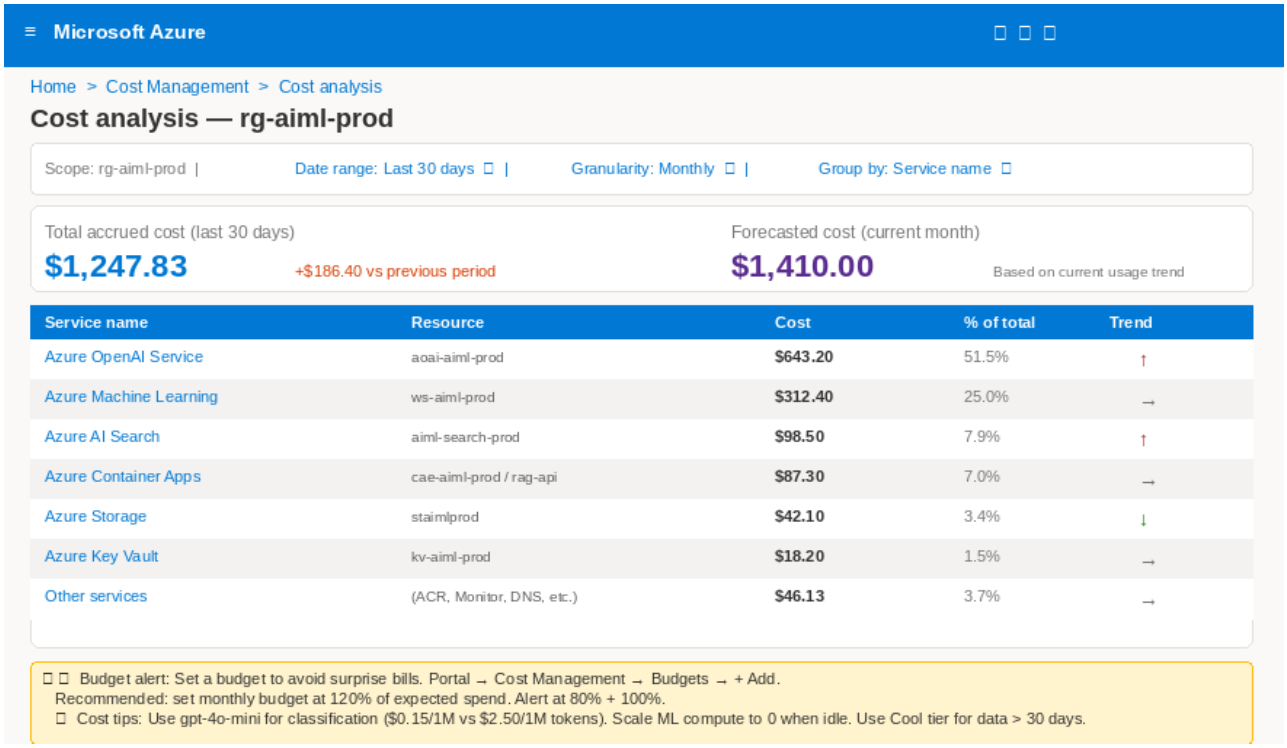


Fig 14.1 — Cost analysis: per-service breakdown with trends and budget alert

Chapter 15: Troubleshooting — Common First-Day Errors

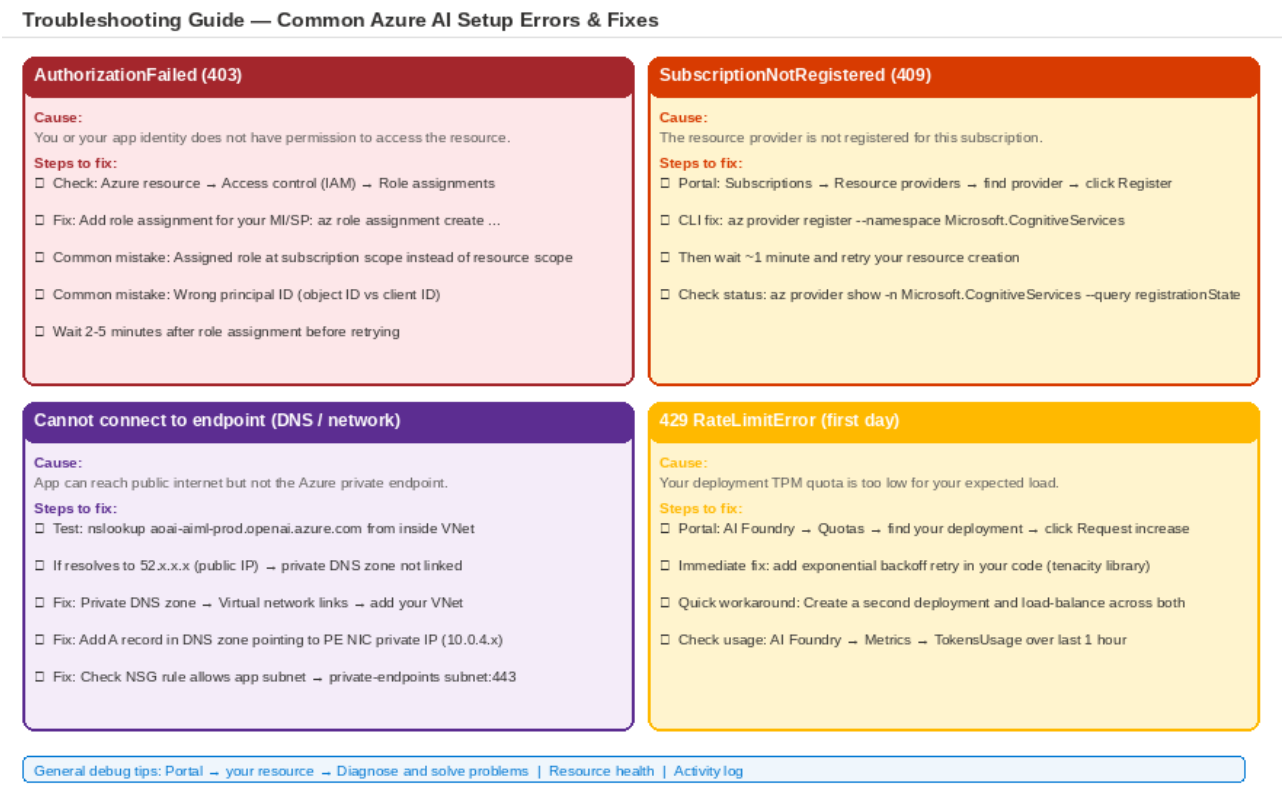


Fig 15.1 — Troubleshooting guide: AuthorizationFailed, SubscriptionNotRegistered, DNS, 429

Error	Cause	Fix
403 AuthorizationFailed	RBAC role not assigned, or wrong scope	Resource → Access control (IAM) → Role assignments → verify your MI is listed with correct role at resource scope
409 SubscriptionNotRegistered	Resource provider not registered	Subscriptions → Resource providers → find → Register → wait 1 minute
DNS returns public IP	Private DNS zone not linked to VNet	Private DNS zones → zone → Virtual network links → add your VNet
429 RateLimitError	TPM quota exceeded	AI Foundry → Quotas → deployment → Request increase; add tenacity retry in code
ResourceNotFound 404	Wrong resource name or wrong subscription	az account show to verify subscription; check exact name in portal

💡 When stuck: Portal → your resource → Diagnose and solve problems — runs automated checks. Also check: Activity log (all recent changes) and Resource health (Azure service-side outages).

PART 5: AI USE CASES — End-to-End Examples

Chapter 16: RAG Chatbot — Enterprise Knowledge Q&A

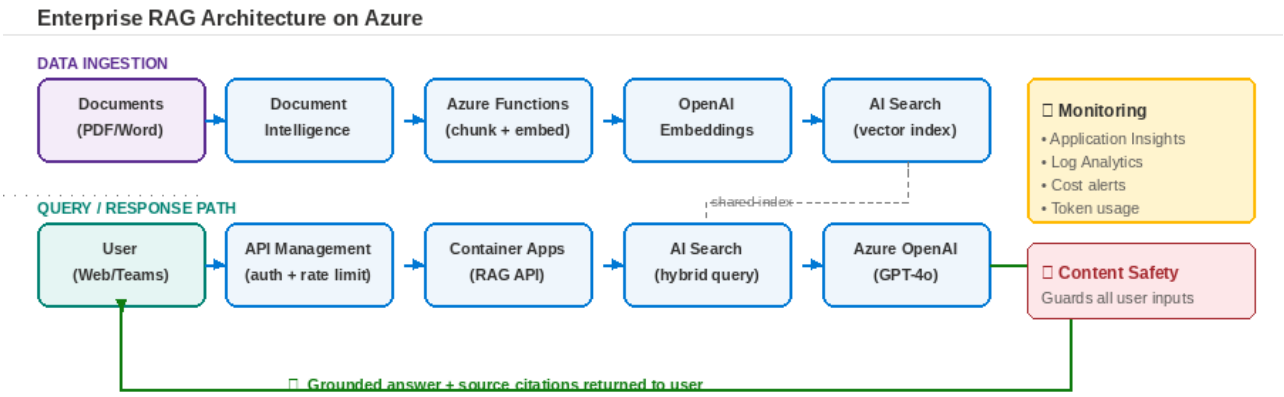


Fig 16.1 — RAG architecture: ingestion path (top) and query/response path (bottom)

Fig 16.2 — RAG ingestion workflow: 7 steps from upload to searchable index

```
async def ingest(blob_url: str, tenant_id: str) -> None:
    """Blob created → extract → chunk → embed → index"""
    text = extract_with_doc_intel(blob_url)
    chunks = [" ".join(text.split()[i:i+400]) for i in range(0, len(text.split()), 340)]
    embeddings = aoai.embeddings.create(model="text-embed-large", input=chunks)
    search.merge_or_upload_documents([
        {"id": f"{blob_url.split('/')[-1]}_{i}",
         "content": c, "embedding": e.embedding,
         "source_url": blob_url, "tenant_id": tenant_id}
        for i,(c,e) in enumerate(zip(chunks, embeddings.data))
    ])

async def rag_query(question: str, tenant_id: str) -> dict:
    """Embed → retrieve → generate → store"""
    q_emb = aoai.embeddings.create(model="text-embed-large",
                                   input=[question]).data[0].embedding

    chunks = list(search.search(
        search_text=question,
```

```
        vector_queries=[VectorizedQuery(vector=q_emb, k_nearest_neighbors=5,
fields="embedding")],
        filter=f"tenant_id eq '{tenant_id}'",   # CRITICAL: data isolation
        query_type="semantic", semantic_configuration_name="default", top=5))

    ctx = "\n\n".join(f"[{i+1}] {r['content']}" for i,r in enumerate(chunks))

    resp = aoai.chat.completions.create(
        model="gpt-4o",
        messages=[
            {"role":"system","content":f"Answer ONLY from context. Cite [1],[2].\n\n{ctx}"},
            {"role":"user","content":question}],
        max_tokens=1500, temperature=0.1)

    return {"answer": resp.choices[0].message.content,
            "sources": [r["source_url"] for r in chunks]}
```

Chapter 17: Document Intelligence Pipeline

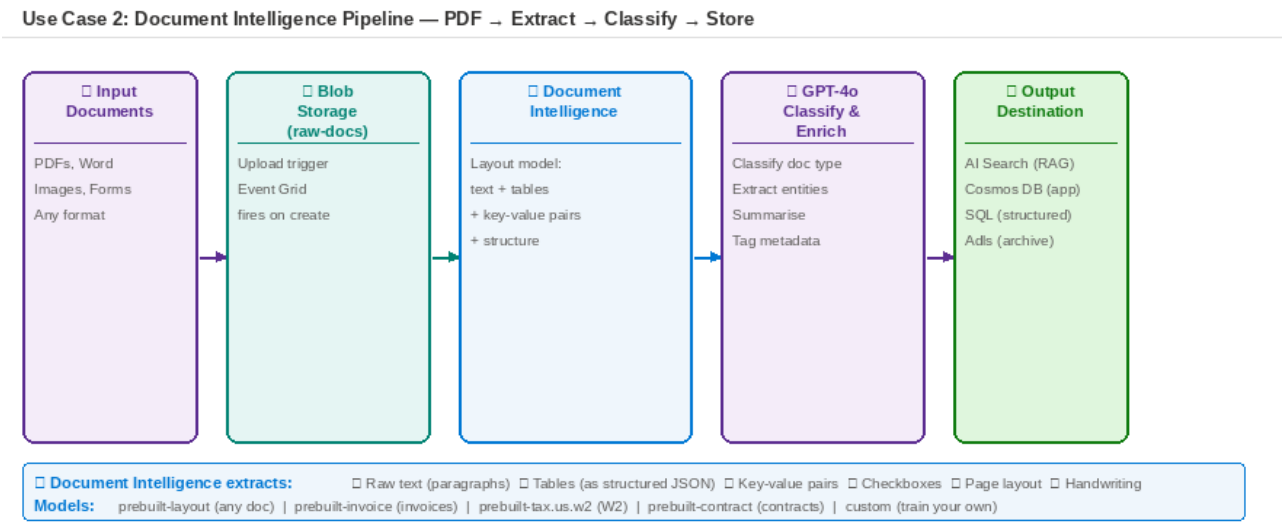


Fig 17.1 — Document pipeline: upload → extract → classify → route to system

```
from azure.ai.documentintelligence import DocumentIntelligenceClient
from pydantic import BaseModel

doc_intel = DocumentIntelligenceClient(os.environ["DOC_INTEL_ENDPOINT"],
DefaultAzureCredential())

class DocAnalysis(BaseModel):
    doc_type: str       # invoice, contract, receipt, form, other
    vendor: str | None; total: float | None
    key_entities: list[str]; confidence: float

async def process_document(blob_url: str) -> dict:
    # 1. Extract with Document Intelligence
    result = doc_intel.begin_analyze_document_from_url("prebuilt-layout", blob_url).result()
    text = " ".join(p.content for p in (result.paragraphs or []))
    kvps = {kv.key.content: kv.value.content for kv in (result.key_value_pairs or []) if
kv.value}

    # 2. Classify with GPT-4o-mini (cheaper for classification)
    resp = aoai.beta.chat.completions.parse(
        model="gpt-4o-mini",
        messages=[{"role":"user","content":f"Classify:\n{text[:3000]}\nKVPs:{kvps}"}],
        response_format=DocAnalysis)
    analysis = resp.choices[0].message.parsed

    # 3. Route based on classification
    if analysis.doc_type == "invoice" and analysis.confidence > 0.85:
        await store_in_accounts_payable(analysis, blob_url)
    else:
        await queue_for_human_review(blob_url, analysis)
    return analysis.model_dump()
```

Chapter 18: AI Customer Support

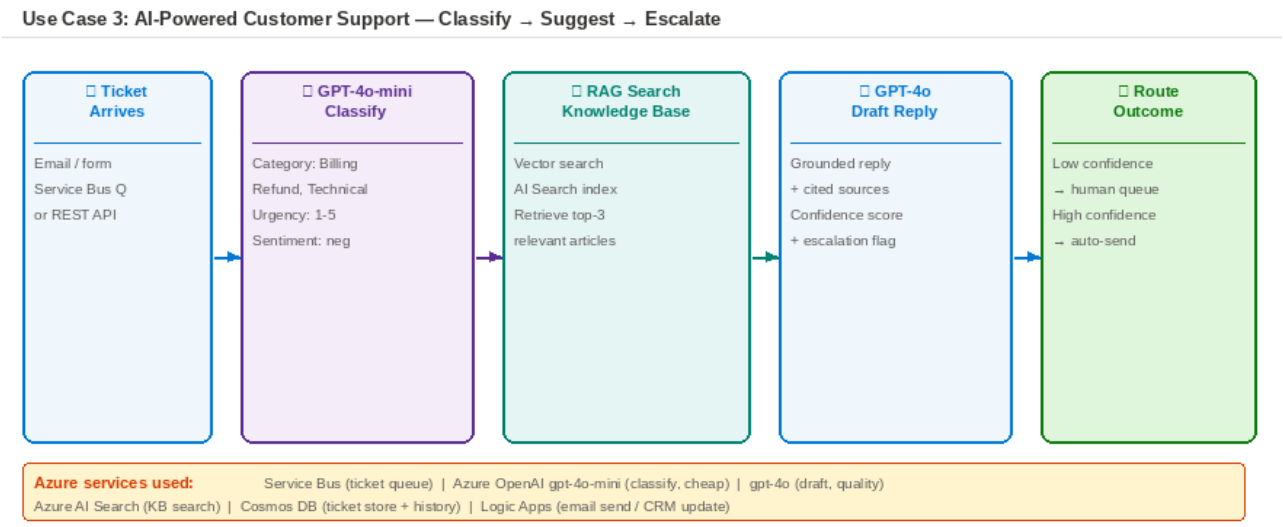


Fig 18.1 — Classify ticket → search KB → draft reply → auto-send or human queue

Chapter 19: Semantic Search

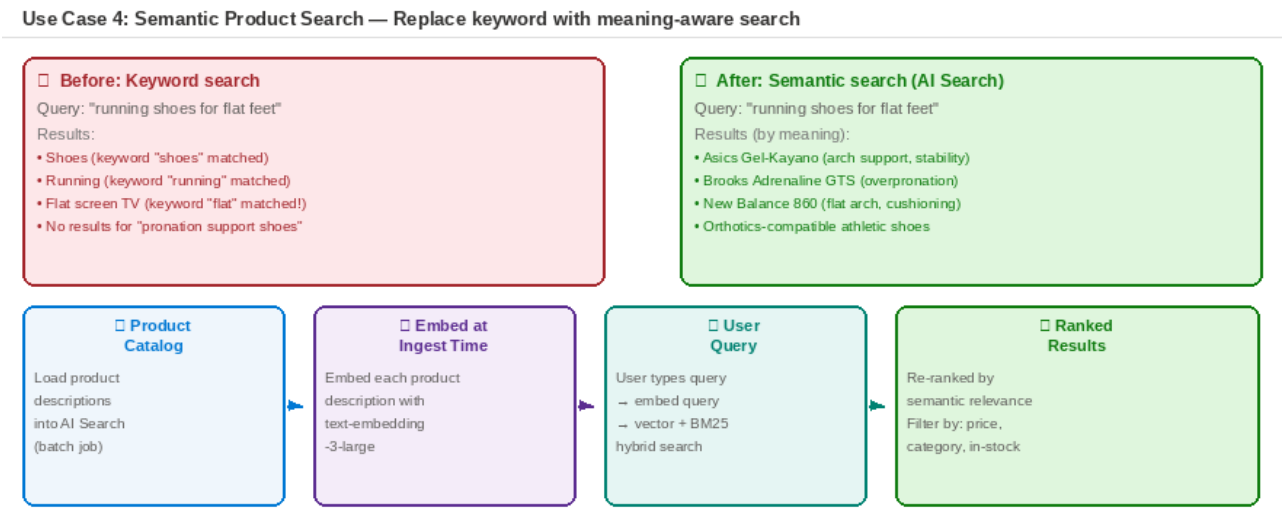


Fig 19.1 — Semantic search: before/after comparison, product embedding + query flow

Chapter 20: Batch Content Generation

```
# OpenAI Batch API — 50% cost saving for async workloads
import json, time

def batch_generate(items: list[dict]) -> dict:
    requests = [{"custom_id": f"item-{i}", "method": "POST",
                 "url": "/v1/chat/completions",
                 "body": {"model": "gpt-4o-mini",
                          "messages": [{"role":"user","content":item["prompt"]}],
                          "max_tokens": 200}}
                for i, item in enumerate(items)]

    jsonl = "\n".join(json.dumps(r) for r in requests)
    f = aoai.files.create(file=("batch.jsonl", jsonl.encode()), purpose="batch")

    batch = aoai.batches.create(
        input_file_id=f.id, endpoint="/v1/chat/completions", completion_window="24h")

    while batch.status not in ["completed","failed","cancelled"]:
        time.sleep(300)  # poll every 5 minutes
        batch = aoai.batches.retrieve(batch.id)

    if batch.output_file_id:
        results = {}
        for line in aoai.files.content(batch.output_file_id).text.split("\n"):
            if line:
                r = json.loads(line)
                results[r["custom_id"]] = r["response"]["body"]["choices"][0]["message"]
["content"]
        return results
```

APPENDICES

Appendix A: CLI Quick Reference

Task	Command
Login	az login && az account show --query "{sub:id, tenant:tenantId}" -o table
Create RG	az group create -n rg-aiml-prod -l eastus2 --tags env=prod team=ai-ml
Register providers	for P in Microsoft.CognitiveServices Microsoft.Search Microsoft.App; do az provider register -n $P; done
Create OpenAI	az cognitiveservices account create -n aoai-aiml-prod -g rg-aiml-prod --kind OpenAI --sku S0 -l eastus2 --custom-domain aoai-aiml-prod
Create AI Search	az search service create -n aiml-search-prod -g rg-aiml-prod --sku standard
Create Key Vault	az keyvault create -n kv-aiml-prod -g rg-aiml-prod --enable-rbac-authorization --enable-purge-protection
Create ACR	az acr create -n acraimlprod -g rg-aiml-prod --sku Premium --admin-enabled false
Build + push image	az acr build --registry acraimlprod --image rag-api:$(git rev-parse --short HEAD) .
Create Container App	az containerapp create -n rag-api -g rg-aiml-prod --environment cae-aiml-prod --image acraimlprod.azurecr.io/rag-api:latest --target-port 8000 --ingress external --system-assigned
Assign RBAC role	az role assignment create --role "Cognitive Services OpenAI User" --assignee <principal-id> --scope <resource-id>
List role assignments	az role assignment list --assignee <principal-id> --all -o table
Test PE DNS	nslookup aoai-aiml-prod.openai.azure.com (should return 10.0.4.x)

Appendix B: Python SDK Quick Reference

```
# All imports for an Azure AI Python project
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
from openai import AzureOpenAI
from azure.search.documents import SearchClient
from azure.search.documents.models import VectorizedQuery
from azure.keyvault.secrets import SecretClient
from azure.storage.blob import BlobServiceClient
from azure.ai.documentintelligence import DocumentIntelligenceClient
from azure.monitor.opentelemetry import configure_azure_monitor
import os

# One credential — auto-detects MI (Azure), az login (local), env vars (CI/CD)
cred = DefaultAzureCredential()

# Azure OpenAI — keyless auth (NEVER use openai.api_key in production)
aoai = AzureOpenAI(
    azure_endpoint=os.environ["OPENAI_ENDPOINT"],
    azure_ad_token_provider=get_bearer_token_provider(
        cred, "https://cognitiveservices.azure.com/.default"),
    api_version="2024-08-01-preview")

# AI Search
search = SearchClient(os.environ["SEARCH_ENDPOINT"], "rag-docs-index", cred)

# Key Vault
kv = SecretClient("https://kv-aiml-prod.vault.azure.net/", cred)

# Blob Storage
blob = BlobServiceClient("https://staimlprod.blob.core.windows.net", cred)

# Document Intelligence
doc = DocumentIntelligenceClient(os.environ["DOC_INTEL_ENDPOINT"], cred)

# Observability (call once at startup)
configure_azure_monitor(connection_string=os.environ["APPLICATIONINSIGHTS_CONNECTION_STRING"])
```

Appendix C: Service Relationships Map

Fig C.1 — How all Azure AI services relate: identity, networking, compute, data, observability

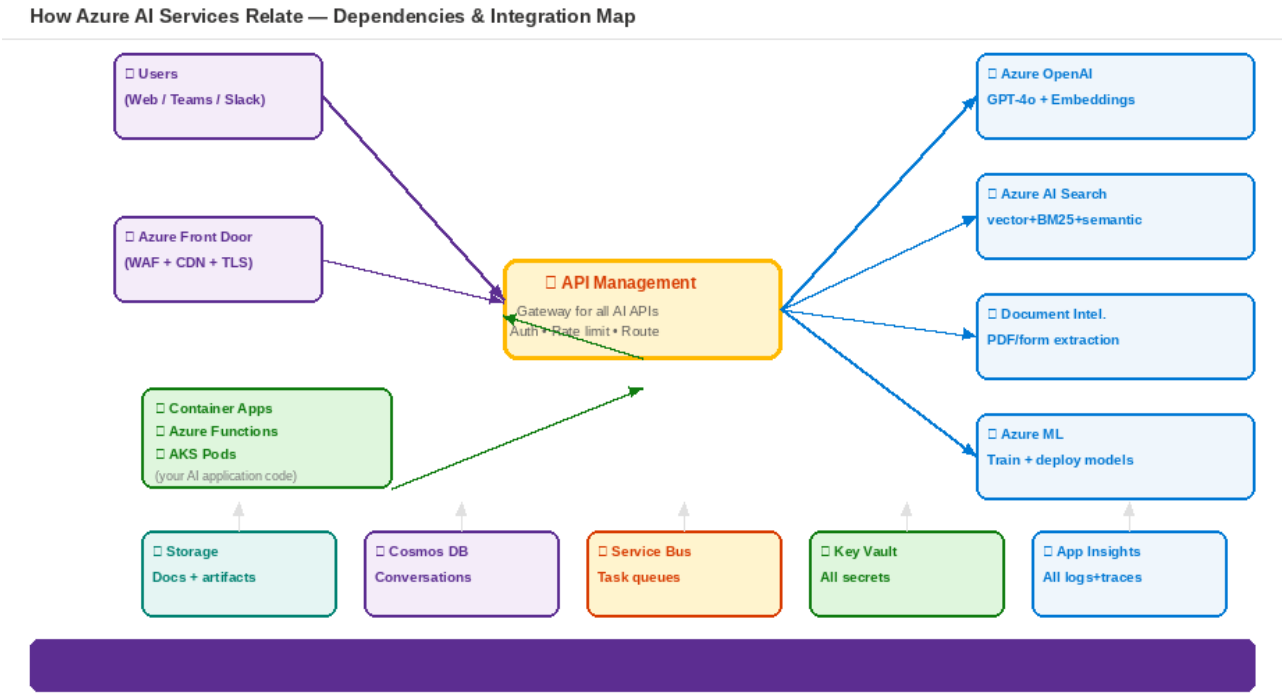


Fig C.2 — Service integration map showing dependencies and connection patterns

— End of Handbook (Seventh Edition) —